

M66 Genetic Algorithm Service Optimization

This notebook rebuilds and validates the M66 reference simulation conditions, then applies the same Genetic Algorithm (GA) service-allocation method to Baseline, High-Demand, Reduced-Service, and Combined conditions.

The optimizer redistributes each scenario's fixed Morning and Evening trip budget across the five study hours. Candidate buses are evenly spaced within each hour, downstream arrivals use hour-specific median GTFS travel offsets, and candidate starting occupancy and alighting assignments are recalculated from the same scenario inputs used by the simulation.

The GA search uses the first five matched replications, optimizer seeds 101, 202, and 303, and a budget of 10 unique service plans per optimizer run. Final plan selection is based on the full matched validation-replication set and requires the selected plan's mean final backlog to be no worse than the corresponding reference service.

Reduced-Service and Combined use fixed period budgets of 38 Morning trips and 39 Evening trips with hourly feasibility bounds of 4 to 12 trips. Combined uses the High-Demand passenger streams together with the Reduced-Service service condition.

Notebook Role

Purpose: Apply the selected Genetic Algorithm consistently to Baseline, High-Demand, Reduced-Service, and Combined conditions and validate the selected plans against their matching references.

Run order: 4 of 4

Main outputs: GA run results, selected plans, safeguards, precision checks, stop-level results, paired 95% confidence intervals, and the GA output manifest.

Technical reference: `../docs/TECHNICAL_DOCUMENTATION.md`

```
In [1]: from pathlib import Path
import heapq
import ast
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import t
```

Technical Reference

Detailed data, simulation, optimization, validation, and methodological references are documented in

```
../docs/TECHNICAL_DOCUMENTATION.md .
```

1. Setup and Configuration

```
In [2]: # Locate the MODESIM_Project directory from common notebook launch locations.
working_dir = Path.cwd()

project_candidates = [
    working_dir,
    working_dir.parent,
    working_dir / "MODESIM_Project",
]

project_dir = next(
    (
        path
        for path in project_candidates
        if (path / "code").exists() and (path / "data").exists()
    ),
    None,
)

if project_dir is None:
    raise FileNotFoundError(
        "Could not locate MODESIM_Project. "
        "Run the notebook from the repository root or MODESIM_Project/code."
    )

raw_dir = project_dir / "data" / "raw"
```

```

processed_dir = project_dir / "data" / "processed"
gtfs_dir = raw_dir / "gtfs_m"

figures_dir = project_dir / "output" / "figures"
tables_dir = project_dir / "output" / "tables"

clean_file = processed_dir / "MTA_M66_Selected_Stops_2025_CLEAN.csv"
raw_file = raw_dir / "MTA_M66_Eastbound_2025_RAW.csv"

figures_dir.mkdir(parents=True, exist_ok=True)
tables_dir.mkdir(parents=True, exist_ok=True)

required_inputs = [
    clean_file,
    raw_file,
    gtfs_dir / "trips.txt",
    gtfs_dir / "stop_times.txt",
]

missing_inputs = [path for path in required_inputs if not path.exists()]

if missing_inputs:
    missing_list = "\n".join(f"- {path}" for path in missing_inputs)
    raise FileNotFoundError(
        f"Required simulation inputs are missing:\n{missing_list}"
    )

print("Project directory located successfully.")
print("Required simulation inputs found.")

```

Project directory located successfully.
 Required simulation inputs found.

```

In [3]: # Core simulation configuration
        bus_capacity = 60

        morning_hours = [6, 7, 8, 9, 10]
        evening_hours = [15, 16, 17, 18, 19]
        study_hours = morning_hours + evening_hours

        selected_stops = [6, 7, 8, 9, 10]

```

```

master_seed = 2026

# Scenario parameters
demand_growth_factor = 1.40
trips_removed_per_hour = 1

# Replication and precision parameters
precision_start_replications = 30
precision_increment = 10
precision_max_replications = 50
precision_confidence_level = 0.95

```

2. Load Passenger Demand

```

In [4]: # Load the cleaned M66 selected-stop dataset used as the passenger-demand input
# for all simulation scenarios.
df = pd.read_csv(clean_file)

# Convert the date and hour fields into consistent datetime/hour values so the
# records can be checked against the defined study periods.
df["Date"] = pd.to_datetime(df["Date"])
df["Hour"] = pd.to_datetime(df["Hour"]).dt.hour

# Collect the study hours and stop sequences that are actually present in the
# processed dataset.
actual_hours = sorted(df["Hour"].unique().tolist())
actual_stops = sorted(df["Stop Sequence"].unique().tolist())

# Fail early if the loaded dataset does not contain exactly the study hours
# expected by the simulation configuration.
if actual_hours != study_hours:
    raise ValueError(
        f"Unexpected study hours. Expected {study_hours}, found {actual_hours}."
    )

# Fail early if the loaded dataset contains a different set of modeled stops.
# This prevents the simulation from silently running with the wrong input data.
if actual_stops != selected_stops:

```

```

raise ValueError(
    f"Unexpected selected stops. Expected {selected_stops}, "
    f"found {actual_stops}."
)

# Display a compact input summary so a reviewer can confirm that the intended
# processed dataset was loaded before the simulation continues.
print(f"Cleaned rows: {len(df):,}")
print(f"Study hours: {actual_hours}")
print(f"Selected stops: {actual_stops}")

```

Cleaned rows: 12,814

Study hours: [6, 7, 8, 9, 10, 15, 16, 17, 18, 19]

Selected stops: [6, 7, 8, 9, 10]

```

In [5]: # Reference: pandas DataFrame.groupby()
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html
#
# Representative mean Boardings/Alightings by stop-hour, and the baseline
# whole-number Passenger Count used by the simulation. Identical rounding
# rule as the baseline notebook: max(0, floor(p + 0.5)).

demand_table = (
    df.groupby(["Stop Sequence", "Stop Name", "Hour"])[["Boardings", "Alightings"]]
    .mean()
    .reset_index()
    .sort_values(["Stop Sequence", "Hour"])
    .reset_index(drop=True)
)

baseline_demand = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Boardings"]
].copy()

baseline_demand["Passenger Count"] = np.maximum(
    0,
    np.floor(baseline_demand["Boardings"] + 0.5)
).astype(int)

print("Baseline passenger total per replication:", baseline_demand["Passenger Count"].sum())
baseline_demand.head(10)

```

Baseline passenger total per replication: 776

Out[5]:

	Stop Sequence	Stop Name	Hour	Boardings	Passenger Count
0	6 W 65 ST/CENTRAL PARK WEST		6	7.543307	8
1	6 W 65 ST/CENTRAL PARK WEST		7	25.386100	25
2	6 W 65 ST/CENTRAL PARK WEST		8	40.393822	40
3	6 W 65 ST/CENTRAL PARK WEST		9	24.162791	24
4	6 W 65 ST/CENTRAL PARK WEST		10	18.968750	19
5	6 W 65 ST/CENTRAL PARK WEST		15	33.683398	34
6	6 W 65 ST/CENTRAL PARK WEST		16	28.030888	28
7	6 W 65 ST/CENTRAL PARK WEST		17	31.360153	31
8	6 W 65 ST/CENTRAL PARK WEST		18	22.000000	22
9	6 W 65 ST/CENTRAL PARK WEST		19	11.316206	11

3. Build High-Demand Passenger Demand

The High-Demand scenario increases boarding demand by 40% at each modeled stop-hour while keeping the same stops, study hours, and Baseline service conditions.

For a Baseline stop-hour boarding value (p), the High-Demand passenger count is calculated as:

$$[p_{\text{HD}}] = \max(0, \lfloor 1.40p + 0.5 \rfloor)$$

The `+0.5` before applying the floor operation implements the same half-up whole-passenger rounding rule used for the Baseline demand inputs. Because rounding is performed separately for every stop-hour cell, the realized total increase may differ slightly from exactly 40%.

For matched scenario comparisons, the original Baseline passengers are preserved. Only the additional passengers required to reach the High-Demand count are generated separately using a deterministic random sub-stream. This allows the Baseline portion of the passenger stream to remain unchanged between Baseline and High-Demand replications.

```
In [6]: # Build the High-Demand stop-hour table from the same demand inputs used
# for the Baseline scenario.
high_demand_demand = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Boardings"]
].copy()

# Increase boarding demand by the configured High-Demand multiplier.
high_demand_demand["Scaled Boardings"] = (
    high_demand_demand["Boardings"] * demand_growth_factor
)

# Convert scaled boarding demand to whole passengers using the same
# half-up rounding rule used for the Baseline passenger counts.
high_demand_demand["Passenger Count"] = np.maximum(
    0,
    np.floor(high_demand_demand["Scaled Boardings"] + 0.5)
).astype(int)

# Compare the Baseline and High-Demand totals and report the realized
# demand increase after stop-hour rounding.
```

```

baseline_total = baseline_demand["Passenger Count"].sum()
high_demand_total = high_demand_demand["Passenger Count"].sum()

print("Baseline passenger total per replication:", baseline_total)
print("High-Demand passenger total per replication:", high_demand_total)
print(
    "Realized demand growth:",
    round(high_demand_total / baseline_total, 4)
)

high_demand_demand.head(10)

```

Baseline passenger total per replication: 776

High-Demand passenger total per replication: 1083

Realized demand growth: 1.3956

Out[6]:

	Stop Sequence	Stop Name	Hour	Boardings	Scaled Boardings	Passenger Count
0	6 W 65 ST/CENTRAL PARK WEST		6	7.543307	10.560630	11
1	6 W 65 ST/CENTRAL PARK WEST		7	25.386100	35.540541	36
2	6 W 65 ST/CENTRAL PARK WEST		8	40.393822	56.551351	57
3	6 W 65 ST/CENTRAL PARK WEST		9	24.162791	33.827907	34
4	6 W 65 ST/CENTRAL PARK WEST		10	18.968750	26.556250	27
5	6 W 65 ST/CENTRAL PARK WEST		15	33.683398	47.156757	47
6	6 W 65 ST/CENTRAL PARK WEST		16	28.030888	39.243243	39
7	6 W 65 ST/CENTRAL PARK WEST		17	31.360153	43.904215	44
8	6 W 65 ST/CENTRAL PARK WEST		18	22.000000	30.800000	31
9	6 W 65 ST/CENTRAL PARK WEST		19	11.316206	15.842688	16

```

In [7]: # Calculate the additional passengers required in each stop-hour cell.
# The Baseline passenger counts are kept separately for matched scenario runs.
additional_demand = baseline_demand[
    ["Stop Sequence", "Stop Name", "Hour", "Passenger Count"]
].rename(

```

```

        columns={"Passenger Count": "Baseline Passenger Count"}
    ).merge(
        high_demand_demand[
            ["Stop Sequence", "Stop Name", "Hour", "Passenger Count"]
        ].rename(
            columns={"Passenger Count": "High-Demand Passenger Count"}
        ),
        on=["Stop Sequence", "Stop Name", "Hour"]
    )

    additional_demand["Additional Passenger Count"] = (
        additional_demand["High-Demand Passenger Count"]
        - additional_demand["Baseline Passenger Count"]
    )

    # Verify that applying the High-Demand multiplier never decreases the
    # passenger count of any Baseline stop-hour cell.
    assert (additional_demand["Additional Passenger Count"] >= 0).all(), (
        "Every High-Demand stop-hour count must be greater than or equal "
        "to its Baseline count."
    )

    # Report the number of new passengers that must be generated in addition
    # to the unchanged Baseline passenger stream.
    additional_passenger_total = additional_demand[
        "Additional Passenger Count"
    ].sum()

    print(
        "Additional passengers to generate per replication:",
        additional_passenger_total
    )

    additional_demand.head(10)

```

Additional passengers to generate per replication: 307

Out[7]:

	Stop Sequence	Stop Name	Hour	Baseline Passenger Count	High-Demand Passenger Count	Additional Passenger Count
0	6	W 65 ST/CENTRAL PARK WEST	6	8	11	3
1	6	W 65 ST/CENTRAL PARK WEST	7	25	36	11
2	6	W 65 ST/CENTRAL PARK WEST	8	40	57	17
3	6	W 65 ST/CENTRAL PARK WEST	9	24	34	10
4	6	W 65 ST/CENTRAL PARK WEST	10	19	27	8
5	6	W 65 ST/CENTRAL PARK WEST	15	34	47	13
6	6	W 65 ST/CENTRAL PARK WEST	16	28	39	11
7	6	W 65 ST/CENTRAL PARK WEST	17	31	44	13
8	6	W 65 ST/CENTRAL PARK WEST	18	22	31	9
9	6	W 65 ST/CENTRAL PARK WEST	19	11	16	5

4. Load GTFS Reference Service

The reference service is taken from the frozen MTA static GTFS data included with the project.

The simulation uses weekday eastbound M66 trips on the full-route shape `M660030`. Short-turn trips are excluded so that every modeled trip serves all five selected stops, Stop Sequences 6 through 10.

A trip is assigned to a study hour using its scheduled arrival time at Stop Sequence 6, the first modeled stop. The published GTFS stop times are retained for the reference scenarios.

```
In [8]: # Load the GTFS trip and stop-time tables used to construct the
# reference M66 service schedule.
trips = pd.read_csv(gtfs_dir / "trips.txt")
stop_times = pd.read_csv(gtfs_dir / "stop_times.txt")

# Keep weekday eastbound M66 trips that use the selected full-route shape.
# This excludes short-turn service that does not follow the full study segment.
m66_trips = trips[
    (trips["route_id"] == "M66")
    & (trips["service_id"] == "MQ_C6-Weekday")
    & (trips["direction_id"] == 0)
    & (trips["shape_id"] == "M660030")
].copy()

if m66_trips.empty:
    raise ValueError(
        "No weekday eastbound M66 full-route trips were found in the GTFS input."
    )

print("Full-route weekday trips:", len(m66_trips))
```

Full-route weekday trips: 138

```
In [9]: # Keep GTFS stop-time records for the five modeled stops.
selected_stop_times = stop_times[
    stop_times["trip_id"].isin(m66_trips["trip_id"])
    & stop_times["stop_sequence"].between(6, 10)
].copy()

# Use Stop Sequence 6 as the reference point for assigning each trip
# to a study hour.
first_stop_times = selected_stop_times[
    selected_stop_times["stop_sequence"] == 6
].copy()

first_stop_times["Hour"] = (
    first_stop_times["arrival_time"]
```

```

        .str.split(":")
        .str[0]
        .astype(int)
    )

    # Identify trips whose Stop 6 arrival occurs within the defined
    # Morning or Evening study hours.
    study_trip_ids = first_stop_times[
        first_stop_times["Hour"].isin(study_hours)
    ]["trip_id"]

    study_stop_times = selected_stop_times[
        selected_stop_times["trip_id"].isin(study_trip_ids)
    ].copy()

    # Convert GTFS arrival times to minutes after midnight for use by the
    # simulation and the Reduced-Service trip-removal rule.
    def time_to_minutes(time_text):
        """Convert a GTFS HH:MM:SS time value to minutes after midnight."""
        hours, minutes, seconds = map(int, time_text.split(":"))
        return (hours * 60) + minutes + (seconds / 60)

    study_stop_times["arrival_minutes"] = (
        study_stop_times["arrival_time"].apply(time_to_minutes)
    )

    first_stop_times["Arrival Minutes"] = (
        first_stop_times["arrival_time"].apply(time_to_minutes)
    )

    # Count reference trips according to their scheduled Stop 6 arrival hour.
    trips_per_hour = (
        first_stop_times[
            first_stop_times["Hour"].isin(study_hours)
        ]
        .groupby("Hour")
        .size()
        .reindex(study_hours, fill_value=0)
    )

```

```

# Verify that each selected trip has one stop-time record at all five
# modeled stops.
expected_stop_events = study_trip_ids.nunique() * len(selected_stops)

if len(study_stop_times) != expected_stop_events:
    raise ValueError(
        "The GTFS study service does not contain exactly one record "
        "for every modeled stop on every selected trip."
    )

print("Study-period trips:", study_trip_ids.nunique())
print("Study-period stop events:", len(study_stop_times))
print("\nBaseline trips per hour:")
print(trips_per_hour)

```

Study-period trips: 87

Study-period stop events: 435

Baseline trips per hour:

Hour

6	5
7	12
8	12
9	8
10	6
15	8
16	12
17	11
18	8
19	5

dtype: int64

In [10]: # Build the hourly Baseline service plan from the reference GTFS trip counts.

```

baseline_service_plan = trips_per_hour.reset_index(name="Trips")

baseline_service_plan["Period"] = np.where(
    baseline_service_plan["Hour"] < 12,
    "Morning",
    "Evening"
)

```

```
# Calculate the fixed Morning and Evening trip budgets used for  
# reference-service comparisons.  
service_budget = (  
    baseline_service_plan  
        .groupby("Period")["Trips"]  
        .sum()  
        .to_dict()  
)  
  
print(  
    "Baseline Morning service budget:",  
    service_budget["Morning"]  
)  
print(  
    "Baseline Evening service budget:",  
    service_budget["Evening"]  
)  
  
baseline_service_plan
```

Baseline Morning service budget: 43

Baseline Evening service budget: 44

Out[10]:

	Hour	Trips	Period
0	6	5	Morning
1	7	12	Morning
2	8	12	Morning
3	9	8	Morning
4	10	6	Morning
5	15	8	Evening
6	16	12	Evening
7	17	11	Evening
8	18	8	Evening
9	19	5	Evening

5. Build Reduced-Service Reference Schedule

The Reduced-Service scenario removes exactly one published GTFS trip from every study hour. Passenger demand remains at the Baseline level.

Trip removal follows a deterministic closest-to-midpoint rule:

1. For each study hour, scheduled trips are identified using their Stop Sequence 6 arrival times.
2. The midpoint of the hour is defined as `HH:30`.
3. The trip with the smallest absolute time difference from the midpoint is removed.
4. If two trips are equally close, the earlier scheduled trip is removed.
5. The selected `trip_id` is removed from all five modeled stops so that no partial trip remains.

The remaining trips retain their original published GTFS arrival times. No synthetic service times are created for the Reduced-Service reference schedule.

Removing one trip from each of the five Morning hours reduces the Morning service budget from 43 to 38 trips. Removing one trip from each of the five Evening hours reduces the Evening service budget from 44 to 39 trips.

```
In [11]: # Select one trip for removal from each study hour using the
# deterministic closest-to-midpoint rule.
removed_records = []

for hour in study_hours:
    hour_trips = first_stop_times[
        first_stop_times["Hour"] == hour
    ].copy()

    if hour_trips.empty:
        raise ValueError(
            f"No reference trip is available for study hour {hour}."
        )

    hour_midpoint_minutes = (hour * 60) + 30

    # Calculate each trip's absolute distance from the midpoint
    # of its assigned study hour.
    hour_trips["Distance to Midpoint"] = (
        hour_trips["Arrival Minutes"] - hour_midpoint_minutes
    ).abs()

    # Sort by midpoint distance first and scheduled arrival time second.
    # The second key makes the earlier trip win if a tie occurs.
    hour_trips_sorted = hour_trips.sort_values(
        ["Distance to Midpoint", "Arrival Minutes"]
    ).reset_index(drop=True)

    min_distance = hour_trips_sorted[
        "Distance to Midpoint"
    ].iloc[0]

    is_tie = (
        hour_trips_sorted["Distance to Midpoint"] == min_distance
    ).sum() > 1

    removed_trip = hour_trips_sorted.iloc[0]
```

```

removed_records.append({
    "Hour": hour,
    "Period": "Morning" if hour < 12 else "Evening",
    "Baseline Trips": len(hour_trips),
    "Hour Midpoint (minutes)": hour_midpoint_minutes,
    "Removed trip_id": removed_trip["trip_id"],
    "Removed Stop 6 Arrival Time": removed_trip["arrival_time"],
    "Distance to Midpoint (minutes)": round(
        float(min_distance), 3
    ),
    "Tie at Minimum Distance": bool(is_tie)
})

removed_trips_df = pd.DataFrame(removed_records)
removed_trip_ids = set(removed_trips_df["Removed trip_id"])

# Confirm that one unique trip was selected for every study hour.
if len(removed_trip_ids) != len(study_hours):
    raise ValueError(
        "Reduced-Service construction did not select exactly "
        "one unique trip per study hour."
    )

print(
    "Trips removed:",
    len(removed_trip_ids),
    f"(expected: {len(study_hours)})"
)
print(
    "Any ties encountered:",
    removed_trips_df["Tie at Minimum Distance"].any()
)

removed_trips_df

```

Trips removed: 10 (expected: 10)

Any ties encountered: False

Out[11]:

	Hour	Period	Baseline Trips	Hour Midpoint (minutes)	Removed trip_id	Removed Stop 6 Arrival Time	Distance to Midpoint (minutes)	Tie at Minimum Distance
0	6	Morning	5	390	MQ_C6-Weekday-039100_M66_502	06:34:54	4.900	False
1	7	Morning	12	450	MQ_C6-Weekday-044500_M66_509	07:29:54	0.100	False
2	8	Morning	12	510	MQ_C6-Weekday-050600_M66_509	08:31:37	1.617	False
3	9	Morning	8	570	MQ_C6-Weekday-056800_M66_507	09:33:37	3.617	False
4	10	Morning	6	630	MQ_C6-Weekday-062800_M66_513	10:33:48	3.800	False
5	15	Evening	8	930	MQ_C6-Weekday-092300_M66_518	15:28:48	1.200	False
6	16	Evening	12	990	MQ_C6-Weekday-098300_M66_516	16:28:48	1.200	False
7	17	Evening	11	1050	MQ_C6-Weekday-104300_M66_515	17:28:37	1.383	False
8	18	Evening	8	1110	MQ_C6-Weekday-110200_M66_519	18:27:37	2.383	False
9	19	Evening	5	1170	MQ_C6-Weekday-116400_M66_522	19:30:16	0.267	False

In [12]:

```
# Remove the selected trip IDs from all five modeled stops while keeping
# the published GTFS times of every remaining trip unchanged.
reduced_study_stop_times = study_stop_times[
    ~study_stop_times["trip_id"].isin(removed_trip_ids)
].copy()

reduced_first_stop_times = first_stop_times[
    ~first_stop_times["trip_id"].isin(removed_trip_ids)
]
```

```

].copy()

# Count the remaining reference trips in each study hour.
reduced_trips_per_hour = (
    reduced_first_stop_times[
        reduced_first_stop_times["Hour"].isin(study_hours)
    ]
    .groupby("Hour")
    .size()
)

reduced_service_plan = (
    reduced_trips_per_hour
    .reindex(study_hours, fill_value=0)
    .reset_index()
)

reduced_service_plan.columns = ["Hour", "Trips"]

reduced_service_plan["Period"] = np.where(
    reduced_service_plan["Hour"] < 12,
    "Morning",
    "Evening"
)

# Calculate the Morning and Evening service budgets after trip removal.
reduced_service_budget = (
    reduced_service_plan
    .groupby("Period")["Trips"]
    .sum()
    .to_dict()
)

# Compare the hourly reference and Reduced-Service trip counts.
trip_loss_per_hour = (
    baseline_service_plan.set_index("Hour")["Trips"]
    - reduced_service_plan.set_index("Hour")["Trips"]
)

assert (
    trip_loss_per_hour == trips_removed_per_hour

```

```
).all(), (  
    "Every study hour must lose exactly one trip "  
    "under the Reduced-Service rule."  
)  
  
print(  
    "Reduced-Service Morning service budget:",  
    reduced_service_budget["Morning"]  
)  
print(  
    "Reduced-Service Evening service budget:",  
    reduced_service_budget["Evening"]  
)  
print("Confirmed: every study hour lost exactly one trip.")  
  
reduced_service_plan
```

Reduced-Service Morning service budget:

38

Reduced-Service Evening service budget: 39

Confirmed: every study hour lost exactly one trip.

Out[12]:

	Hour	Trips	Period
0	6	4	Morning
1	7	11	Morning
2	8	11	Morning
3	9	7	Morning
4	10	5	Morning
5	15	7	Evening
6	16	11	Evening
7	17	10	Evening
8	18	7	Evening
9	19	4	Evening

In [13]: *# Create a compact comparison of the Baseline and Reduced-Service
trip counts for every study hour.*

```
hourly_trip_counts = baseline_service_plan[
    ["Hour", "Period", "Trips"]
].rename(
    columns={"Trips": "Baseline Trips"}
).merge(
    reduced_service_plan[
        ["Hour", "Trips"]
    ].rename(
        columns={"Trips": "Reduced-Service Trips"}
    ),
    on="Hour"
)

hourly_trip_counts["Trips Removed"] = (
    hourly_trip_counts["Baseline Trips"]
    - hourly_trip_counts["Reduced-Service Trips"]
)
```

Add a final row showing the total number of reference and remaining trips.

```
total_trips_row = pd.DataFrame([{
    "Hour": "Total",
    "Period": "All",
    "Baseline Trips": int(
        hourly_trip_counts["Baseline Trips"].sum()
    ),
    "Reduced-Service Trips": int(
        hourly_trip_counts["Reduced-Service Trips"].sum()
    ),
    "Trips Removed": int(
        hourly_trip_counts["Trips Removed"].sum()
    )
}])
```

```
hourly_trip_counts_with_total = pd.concat(
    [hourly_trip_counts, total_trips_row],
    ignore_index=True
)
```

```
hourly_trip_counts_with_total
```

Out[13]:

	Hour	Period	Baseline Trips	Reduced-Service Trips	Trips Removed
0	6	Morning	5	4	1
1	7	Morning	12	11	1
2	8	Morning	12	11	1
3	9	Morning	8	7	1
4	10	Morning	6	5	1
5	15	Evening	8	7	1
6	16	Evening	12	11	1
7	17	Evening	11	10	1
8	18	Evening	8	7	1
9	19	Evening	5	4	1
10	Total	All	87	77	10

6. Validate Reduced-Service Trip Removal

The Reduced-Service construction is validated independently after the removal rule is applied.

The checks confirm that:

- the selected trip is the trip closest to the midpoint of each study hour;
- the earlier trip is selected if an equal-distance tie occurs;
- each removed `trip_id` is removed from all five modeled stops;
- exactly one trip is removed from every study hour; and
- the resulting service budgets are 38 Morning trips and 39 Evening trips.

The notebook stops with an assertion error if any validation check fails.

```

In [14]: # Recompute the expected removed trip independently for every study hour
# and compare it with the trip selected during schedule construction.
correct_selection = True
tie_earlier_correct = True

for hour in study_hours:
    hour_trips = first_stop_times[
        first_stop_times["Hour"] == hour
    ].copy()

    hour_midpoint_minutes = (hour * 60) + 30

    hour_trips["Distance"] = (
        hour_trips["Arrival Minutes"] - hour_midpoint_minutes
    ).abs()

    min_distance = hour_trips["Distance"].min()

    tied_candidates = hour_trips[
        hour_trips["Distance"] == min_distance
    ].sort_values("Arrival Minutes")

    expected_trip_id = tied_candidates.iloc[0]["trip_id"]

    actual_trip_id = removed_trips_df.loc[
        removed_trips_df["Hour"] == hour,
        "Removed trip_id"
    ].iloc[0]

    if actual_trip_id != expected_trip_id:
        correct_selection = False

    if (
        len(tied_candidates) > 1
        and actual_trip_id != tied_candidates.iloc[0]["trip_id"]
    ):
        tie_earlier_correct = False

# Verify that each selected trip originally contains all five modeled stops

```

```

# and that none of those trip IDs remains in the Reduced-Service schedule.
original_stop_counts = (
    study_stop_times[
        study_stop_times["trip_id"].isin(removed_trip_ids)
    ]
    .groupby("trip_id")["stop_sequence"]
    .nunique()
)

removed_trips_cover_all_stops = (
    len(original_stop_counts) == len(removed_trip_ids)
    and (original_stop_counts == len(selected_stops)).all()
)

no_removed_trips_remain = not reduced_study_stop_times[
    "trip_id"
].isin(removed_trip_ids).any()

full_removal_ok = (
    removed_trips_cover_all_stops
    and no_removed_trips_remain
)

# Verify the final Reduced-Service trip budgets.
expected_reduced_budget = {
    "Morning": (
        service_budget["Morning"]
        - trips_removed_per_hour * len(morning_hours)
    ),
    "Evening": (
        service_budget["Evening"]
        - trips_removed_per_hour * len(evening_hours)
    )
}

service_totals_ok = (
    reduced_service_budget == expected_reduced_budget
)

```

```

# Store the validation results in a compact reviewer-facing table.
validation_records = [
    {
        "Check": (
            "Removed trip is closest to the hour midpoint"
        ),
        "Passed": bool(correct_selection)
    },
    {
        "Check": (
            "Earlier scheduled trip is selected when a midpoint tie occurs"
        ),
        "Passed": bool(tie_earlier_correct)
    },
    {
        "Check": (
            "Removed trip IDs are excluded from all five modeled stops"
        ),
        "Passed": bool(full_removal_ok)
    },
    {
        "Check": (
            "Exactly one trip is removed from every study hour"
        ),
        "Passed": bool(
            (
                trip_loss_per_hour
                == trips_removed_per_hour
            ).all()
        )
    },
    {
        "Check": (
            "Reduced-Service period trip budgets are preserved"
        ),
        "Passed": bool(service_totals_ok)
    }
]

removal_validation_df = pd.DataFrame(validation_records)

```

```

assert removal_validation_df["Passed"].all(), (
    "Reduced-Service trip-removal validation failed. "
    "Review removal_validation_df."
)

print(
    "All Reduced-Service trip-removal validation checks passed."
)

removal_validation_df

```

All Reduced-Service trip-removal validation checks passed.

Out[14]:

	Check	Passed
0	Removed trip is closest to the hour midpoint	True
1	Earlier scheduled trip is selected when a midp...	True
2	Removed trip IDs are excluded from all five mo...	True
3	Exactly one trip is removed from every study hour	True
4	Reduced-Service period trip budgets are preserved	True

7. Build Baseline Occupancy and Alighting Inputs

The simulation represents buses as already carrying passengers when they enter the modeled segment at Stop Sequence 6.

Baseline starting occupancy is estimated from ridership activity at Stop Sequences 1 through 5, which are the M66 stops immediately upstream of the five modeled stops. For each study hour, mean upstream boardings and alightings are used to estimate the upstream net passenger flow. This net flow is divided by the number of scheduled buses in that hour and rounded to a whole-passenger starting occupancy, subject to the modeled bus capacity of 60 passengers.

The High-Demand scenario changes only boarding demand at the five modeled stops. It therefore uses the same upstream starting-occupancy inputs as Baseline rather than assuming an unsupported 40% increase in upstream ridership.

Alighting demand at the modeled stops is also derived from the representative stop-hour MTA ridership values. The rounded stop-hour alighting target is distributed across the buses serving that hour using quotient-and-remainder allocation so that the complete stop-hour target is preserved.

```
In [15]: # Load the raw M66 ridership data used to estimate passenger activity
# at the stops immediately upstream of the modeled segment.
raw_m66 = pd.read_csv(raw_file)

# Convert Date and Hour into the formats used for weekday and
# study-period filtering.
raw_m66["Date"] = pd.to_datetime(raw_m66["Date"])
raw_m66["Hour"] = pd.to_datetime(raw_m66["Hour"]).dt.hour

# Identify Stop Sequences 1 through 5 from the same full-route GTFS
# service used by the simulation.
upstream_gtfs_stops = (
    stop_times[
        stop_times["trip_id"].isin(m66_trips["trip_id"])
        & stop_times["stop_sequence"].between(1, 5)
    ][["stop_id", "stop_sequence"]]
    .drop_duplicates()
    .sort_values("stop_sequence")
)

upstream_stop_ids = (
    upstream_gtfs_stops["stop_id"]
    .astype(int)
    .tolist()
)

# Keep weekday ridership records for the upstream stops during the
# Morning and Evening study hours.
upstream_df = raw_m66[
    (raw_m66["Date"].dt.dayofweek < 5)
    & (raw_m66["Stop ID"].isin(upstream_stop_ids))
    & (raw_m66["Hour"].isin(study_hours))
].copy()

# Remove source-inconsistent records where boardings or alightings are
```

```

# reported even though the source record reports zero trips.
upstream_clean = upstream_df[
    ~(
        (upstream_df["Trips"] == 0)
        & (
            (upstream_df["Boardings"] > 0)
            | (upstream_df["Alightings"] > 0)
        )
    )
].copy()

if upstream_clean.empty:
    raise ValueError(
        "No valid upstream ridership records remain after filtering."
    )

print("Verified upstream Stop IDs:", upstream_stop_ids)
print("Upstream rows after cleaning:", len(upstream_clean))

```

Verified upstream Stop IDs: [403487, 403488, 403489, 403491, 403573]
Upstream rows after cleaning: 10867

```

In [16]: # Calculate representative upstream boardings and alightings for each
# upstream stop and study hour.
upstream_stop_hour = (
    upstream_clean
    .groupby(["Stop ID", "Hour"])[["Boardings", "Alightings"]]
    .mean()
    .reset_index()
)

# Sum the representative upstream activity across Stops 1 through 5
# to estimate the total passenger flow entering the modeled segment.
upstream_hourly_flow = (
    upstream_stop_hour
    .groupby("Hour")[["Boardings", "Alightings"]]
    .sum()
    .reset_index()
    .rename(
        columns={
            "Boardings": "Average Upstream Boardings",

```

```

        "Alightings": "Average Upstream Alightings"
    }
)
)

upstream_hourly_flow["Upstream Net Flow"] = (
    upstream_hourly_flow["Average Upstream Boardings"]
    - upstream_hourly_flow["Average Upstream Alightings"]
)

# Match each study hour to the number of reference buses scheduled
# during that hour.
upstream_hourly_flow["Scheduled Buses"] = (
    upstream_hourly_flow["Hour"].map(trips_per_hour)
)

if upstream_hourly_flow["Scheduled Buses"].isna().any():
    raise ValueError(
        "At least one upstream study hour has no matching GTFS service count."
    )

if (upstream_hourly_flow["Scheduled Buses"] <= 0).any():
    raise ValueError(
        "Starting occupancy cannot be estimated for an hour with no buses."
    )

# Estimate the representative starting occupancy per bus from the
# upstream net flow and the number of buses serving the hour.
upstream_hourly_flow["Initial Occupancy Unrounded"] = (
    upstream_hourly_flow["Upstream Net Flow"]
    / upstream_hourly_flow["Scheduled Buses"]
)

# Convert the estimate to a valid whole-passenger occupancy within the
# modeled 60-passenger bus capacity.
upstream_hourly_flow["Initial Occupancy"] = np.minimum(
    bus_capacity,
    np.maximum(
        0,
        np.floor(
            upstream_hourly_flow["Initial Occupancy Unrounded"] + 0.5

```

```
    )
)
).astype(int)

initial_occupancy_by_hour = (
    upstream_hourly_flow
    .set_index("Hour")["Initial Occupancy"]
    .to_dict()
)

upstream_net_flow_by_hour = (
    upstream_hourly_flow
    .set_index("Hour")["Upstream Net Flow"]
    .to_dict()
)

upstream_hourly_flow[
    [
        "Hour",
        "Upstream Net Flow",
        "Scheduled Buses",
        "Initial Occupancy"
    ]
]
```

Out[16]:

	Hour	Upstream Net Flow	Scheduled Buses	Initial Occupancy
0	6	60.132476	5	12
1	7	163.631926	12	14
2	8	211.704836	12	18
3	9	124.995854	8	16
4	10	91.587024	6	15
5	15	137.208668	8	17
6	16	122.268987	12	10
7	17	118.860856	11	11
8	18	95.938313	8	12
9	19	49.363709	5	10

```
In [17]: # Assign each Baseline reference trip the starting occupancy calculated
# for its Stop 6 arrival hour.
trip_initial_occupancy = first_stop_times[
    first_stop_times["Hour"].isin(study_hours)
][["trip_id", "Hour"]].copy()

trip_initial_occupancy["Initial Occupancy"] = (
    trip_initial_occupancy["Hour"].map(initial_occupancy_by_hour)
)

trip_initial_occupancy["Period"] = np.where(
    trip_initial_occupancy["Hour"] < 12,
    "Morning",
    "Evening"
)

if trip_initial_occupancy["Initial Occupancy"].isna().any():
    raise ValueError(
        "At least one Baseline trip has no starting occupancy assignment."
```

```

    )

print(
    "Baseline trips with starting occupancy:",
    len(trip_initial_occupancy)
)

```

Baseline trips with starting occupancy: 87

```

In [18]: # Build the representative whole-passenger alighting target for every
# modeled stop-hour cell.
hourly_alighting_targets = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Alightings"]
].copy()

hourly_alighting_targets["Alighting Target"] = np.maximum(
    0,
    np.floor(
        hourly_alighting_targets["Alightings"] + 0.5
    )
).astype(int)

# Distribute each stop-hour alighting target across the buses that
# serve that hour while preserving the complete target.
def build_alighting_plan(
    service_stop_times,
    trip_hour_period_table
):
    """Distribute each stop-hour alighting target across eligible buses while preserving the full integer target."""
    # Start from every eligible bus-stop event that can receive an alighting assignment.
    plan = service_stop_times[
        ["trip_id", "stop_sequence", "arrival_minutes"]
    ].copy()

    # Attach each trip's study hour and period before applying the stop-hour target.
    plan = plan.merge(
        trip_hour_period_table[
            ["trip_id", "Hour", "Period"]
        ],
        on="trip_id",

```

```

        how="left"
    )

    plan = plan.rename(
        columns={"stop_sequence": "Stop Sequence"}
    )

    plan = plan.merge(
        hourly_alighting_targets[
            ["Stop Sequence", "Hour", "Alighting Target"]
        ],
        on=["Stop Sequence", "Hour"],
        how="left"
    )

    if plan["Hour"].isna().any():
        raise ValueError(
            "At least one service trip has no study-hour assignment."
        )

    if plan["Alighting Target"].isna().any():
        raise ValueError(
            "At least one service stop-hour has no alighting target."
        )

    plan = plan.sort_values(
        ["Stop Sequence", "Hour", "arrival_minutes"]
    ).reset_index(drop=True)

    # Add the remainder deterministically so the complete integer target is preserved.
    plan["Assigned Alightings"] = 0

    for (stop, hour), group in plan.groupby(
        ["Stop Sequence", "Hour"]
    ):
        target = int(group["Alighting Target"].iloc[0])
        bus_count = len(group)

        quotient = target // bus_count
        remainder = target % bus_count

```

```

        assignments = [quotient + 1] * remainder
        assignments += [quotient] * (
            bus_count - remainder
        )

        plan.loc[
            group.index,
            "Assigned Alightings"
        ] = assignments

    return plan

# Build the Baseline bus-level alighting assignments and Lookup tables
# used by the discrete-event simulation.
bus_alighting_plan = build_alighting_plan(
    study_stop_times,
    trip_initial_occupancy
)

initial_occupancy_lookup = (
    trip_initial_occupancy
    .set_index("trip_id")["Initial Occupancy"]
    .to_dict()
)

alighting_lookup = (
    bus_alighting_plan
    .set_index(
        ["trip_id", "Stop Sequence"]
    )["Assigned Alightings"]
    .to_dict()
)

trip_period_lookup = (
    trip_initial_occupancy
    .set_index("trip_id")["Period"]
    .to_dict()
)

# Verify that distributing alightings across individual buses preserves

```

```

# every original stop-hour target.
alighting_check = (
    bus_alighting_plan
    .groupby(["Stop Sequence", "Hour"])
    .agg(
        Target=("Alighting Target", "first"),
        Assigned=("Assigned Alightings", "sum")
    )
    .reset_index()
)

baseline_alightings_preserved = (
    alighting_check["Target"]
    == alighting_check["Assigned"]
).all()

assert baseline_alightings_preserved, (
    "Baseline bus-level alighting assignments do not preserve "
    "the stop-hour targets."
)

print(
    "Baseline alighting targets preserved:",
    baseline_alightings_preserved
)

```

Baseline alighting targets preserved: True

8. Build Reduced-Service Occupancy and Alighting Inputs

Reduced-Service retains the same representative upstream passenger flow and the same stop-hour alighting targets as Baseline. However, both inputs must be reassigned because fewer buses serve each study hour.

Starting occupancy is therefore recalculated by dividing the same hourly upstream net flow across the remaining Reduced-Service buses. This generally increases the representative starting occupancy per bus because the same estimated upstream passenger flow is carried by fewer vehicles.

The same Baseline stop-hour alighting targets are then redistributed across the remaining buses using the same quotient-and-remainder procedure.

The Combined scenario uses these Reduced-Service occupancy and alighting inputs because it uses the same reduced service schedule. They are not multiplied by 1.40 because the Combined demand increase applies only to boarding demand at the five modeled stops.

```
In [19]: # Recalculate starting occupancy for each study hour using the same
# upstream net flow but the smaller Reduced-Service bus count.
reduced_initial_occupancy_by_hour = {}

for hour in study_hours:
    bus_count = int(
        reduced_trips_per_hour.get(hour, 0)
    )

    if bus_count <= 0:
        raise ValueError(
            f"Reduced-Service has no bus available in study hour {hour}."
        )

    unrounded_occupancy = (
        upstream_net_flow_by_hour[hour]
        / bus_count
    )

    reduced_initial_occupancy_by_hour[hour] = int(
        np.minimum(
            bus_capacity,
            np.maximum(
                0,
                np.floor(unrounded_occupancy + 0.5)
            )
        )
    )

# Assign the recalculated starting occupancy to every remaining
# Reduced-Service trip.
reduced_trip_initial_occupancy = (
    reduced_first_stop_times[
```

```

        reduced_first_stop_times["Hour"].isin(study_hours)
    ][["trip_id", "Hour"]]
    .copy()
)

reduced_trip_initial_occupancy[
    "Initial Occupancy"
] = (
    reduced_trip_initial_occupancy["Hour"]
    .map(reduced_initial_occupancy_by_hour)
)

reduced_trip_initial_occupancy["Period"] = np.where(
    reduced_trip_initial_occupancy["Hour"] < 12,
    "Morning",
    "Evening"
)

if reduced_trip_initial_occupancy[
    "Initial Occupancy"
].isna().any():
    raise ValueError(
        "At least one Reduced-Service trip has no starting "
        "occupancy assignment."
    )

# Build the Reduced-Service occupancy and period lookup tables used
# by the simulation.
reduced_initial_occupancy_lookup = (
    reduced_trip_initial_occupancy
    .set_index("trip_id")["Initial Occupancy"]
    .to_dict()
)

reduced_trip_period_lookup = (
    reduced_trip_initial_occupancy
    .set_index("trip_id")["Period"]
    .to_dict()
)

# Compare Baseline and Reduced-Service starting occupancy by hour.

```

```

occupancy_change = pd.DataFrame({
    "Hour": study_hours,
    "Baseline Initial Occupancy": [
        initial_occupancy_by_hour[hour]
        for hour in study_hours
    ],
    "Reduced-Service Initial Occupancy": [
        reduced_initial_occupancy_by_hour[hour]
        for hour in study_hours
    ]
})

occupancy_change

```

Out[19]:

	Hour	Baseline Initial Occupancy	Reduced-Service Initial Occupancy
--	------	----------------------------	-----------------------------------

0	6	12	15
1	7	14	15
2	8	18	19
3	9	16	18
4	10	15	18
5	15	17	20
6	16	10	11
7	17	11	12
8	18	12	14
9	19	10	12

```

In [20]: # Redistribute the unchanged stop-hour alighting targets across the
# buses that remain in the Reduced-Service schedule.
reduced_bus_alighting_plan = build_alighting_plan(
    reduced_study_stop_times,
    reduced_trip_initial_occupancy
)

```

```

reduced_alighting_lookup = (
    reduced_bus_alighting_plan
    .set_index(
        ["trip_id", "Stop Sequence"]
    )["Assigned Alightings"]
    .to_dict()
)

# Verify that the Reduced-Service bus-level assignments still preserve
# every original stop-hour alighting target.
reduced_alighting_check = (
    reduced_bus_alighting_plan
    .groupby(["Stop Sequence", "Hour"])
    .agg(
        Target=("Alighting Target", "first"),
        Assigned=("Assigned Alightings", "sum")
    )
    .reset_index()
)

reduced_alightings_preserved = (
    reduced_alighting_check["Target"]
    == reduced_alighting_check["Assigned"]
).all()

assert reduced_alightings_preserved, (
    "Reduced-Service bus-level alighting assignments do not "
    "preserve the stop-hour targets."
)

print(
    "Reduced-Service alighting targets preserved:",
    reduced_alightings_preserved
)

```

Reduced-Service alighting targets preserved: True

9. Discrete-Event Simulation Engine

All scenarios use the same discrete-event simulation engine. Scenario differences are supplied through passenger streams, service schedules, starting occupancy, and alighting assignments rather than through separate simulation logic.

Passenger and bus events are processed chronologically. If a bus arrival and passenger arrival occur at exactly the same simulated time, the bus event is processed first.

When a passenger arrives, the passenger joins the FIFO queue at the corresponding stop. When a bus arrives, assigned passengers alight first. The remaining onboard occupancy is then used to determine available capacity, and waiting passengers board in FIFO order up to the modeled 60-passenger capacity.

The simulation records passenger waiting times, queue changes, bus occupancy, completed boardings, and passengers left waiting. Passengers who remain after the last eligible bus are retained as final backlog rather than being served by additional artificial trips.

The same engine produces period-level and stop-level results. It also reports passenger-conservation, waiting-time, and bus-occupancy checks so invalid simulation states can be detected during scenario evaluation.

```
In [21]: # Process passenger arrivals by adding each passenger to the FIFO queue
# at the corresponding modeled stop.
def handle_passenger_arrival(
    event_time,
    stop_sequence,
    passenger_id,
    stop_queues,
    queue_history,
    peak_queue,
    peak_queue_time
):
    """Add one passenger-arrival event to the FIFO queue for the corresponding modeled stop."""
    stop_queues[stop_sequence].append(
        (passenger_id, event_time)
    )

    current_queue = len(stop_queues[stop_sequence])

    # Record the updated queue length after the passenger joins.
    queue_history.append({
        "Time": event_time,
```

```

        "Stop Sequence": stop_sequence,
        "Queue Length": current_queue,
        "Event": "passenger_arrival"
    })

    # Update the stop-level peak queue and the time when it occurs.
    if current_queue > peak_queue[stop_sequence]:
        peak_queue[stop_sequence] = current_queue
        peak_queue_time[stop_sequence] = event_time

# Process each bus arrival using alighting-before-boarding logic and
# enforce the modeled passenger capacity.
def handle_bus_arrival(
    event_time,
    stop_sequence,
    trip_id,
    stop_queues,
    bus_occupancy,
    waiting_records,
    queue_history,
    bus_history,
    occupancy_lookup,
    service_alighting_lookup,
    capacity=bus_capacity
):
    # Initialize the bus occupancy when the trip first enters the
    # modeled five-stop segment.
    """Process one bus arrival using alighting-before-boarding logic and the modeled capacity limit."""
    if trip_id not in bus_occupancy:
        bus_occupancy[trip_id] = int(
            occupancy_lookup[trip_id]
        )

    occupancy_before = bus_occupancy[trip_id]

    # Retrieve the number of passengers assigned to alight at this
    # trip and stop.
    assigned_alightings = int(
        service_alighting_lookup[(trip_id, stop_sequence)]
    )

```

```

# Prevent alightings from exceeding the passengers currently onboard.
actual_alightings = min(
    assigned_alightings,
    bus_occupancy[trip_id]
)

# Apply alightings before calculating the capacity available
# for waiting passengers.
bus_occupancy[trip_id] -= actual_alightings

available_capacity = capacity - bus_occupancy[trip_id]

# Board passengers in FIFO order up to the remaining bus capacity.
boarded = min(
    len(stop_queues[stop_sequence]),
    available_capacity
)

for _ in range(boarded):
    passenger_id, passenger_arrival = stop_queues[
        stop_sequence
    ].pop(0)

    waiting_time = event_time - passenger_arrival

    waiting_records.append({
        "Passenger ID": passenger_id,
        "Stop Sequence": stop_sequence,
        "Trip ID": trip_id,
        "Passenger Arrival": passenger_arrival,
        "Boarding Time": event_time,
        "Waiting Time": waiting_time
    })

    bus_occupancy[trip_id] += boarded

    remaining_queue = len(stop_queues[stop_sequence])

# Record the queue state after the bus finishes boarding.
queue_history.append({

```

```

        "Time": event_time,
        "Stop Sequence": stop_sequence,
        "Queue Length": remaining_queue,
        "Event": "bus_arrival"
    })

    # Record the occupancy and boarding activity for validation
    # and output metrics.
    bus_history.append({
        "Trip ID": trip_id,
        "Stop Sequence": stop_sequence,
        "Time": event_time,
        "Occupancy Before": occupancy_before,
        "Assigned Alightings": assigned_alightings,
        "Actual Alightings": actual_alightings,
        "Boarded": boarded,
        "Occupancy After": bus_occupancy[trip_id],
        "Passengers Left Waiting": remaining_queue
    })

```

```

In [22]: # Build chronological Morning and Evening event queues from the passenger
# stream and service schedule supplied for the current scenario.
#
# Bus events use priority 0 and passenger events use priority 1 so a bus
# is processed first if both events occur at exactly the same time.
def create_period_event_queues(
    passengers,
    service_stop_times,
    trip_period_lookup_table
):
    """Build chronological Morning and Evening event queues for the supplied passenger stream and service plan."""
    morning_events = []
    evening_events = []

    # Add passenger-arrival events to the appropriate study period.
    for _, passenger in passengers.iterrows():
        event = (
            float(passenger["Arrival Minutes"]),
            1,
            "passenger_arrival",

```

```

        int(passenger["Stop Sequence"]),
        passenger["Passenger ID"]
    )

    if passenger["Period"] == "Morning":
        heapq.heappush(morning_events, event)
    else:
        heapq.heappush(evening_events, event)

# Add scheduled bus-arrival events using the service inputs
# supplied for the current scenario.
for _, bus in service_stop_times.iterrows():
    event = (
        float(bus["arrival_minutes"]),
        0,
        "bus_arrival",
        int(bus["stop_sequence"]),
        bus["trip_id"]
    )

    if (
        trip_period_lookup_table[bus["trip_id"]]
        == "Morning"
    ):
        heapq.heappush(morning_events, event)
    else:
        heapq.heappush(evening_events, event)

return morning_events, evening_events

```

```

In [23]: # Run one Morning or Evening simulation period using the shared event
# processing rules and the scenario-specific occupancy and alighting inputs.
#
# The function returns period-level metrics, event histories, stop-level
# results, and validity checks without changing the queueing rules by scenario.
def run_service_simulation(
    period,
    events,
    generated_passengers,
    replication,

```

```

seed,
occupancy_lookup,
service_alighting_lookup
):
    # Define the official boundaries of the requested study period.
    """Run the event-driven queue simulation for one study period and return performance, history, and validation outputs."""
    if period == "Morning":
        start_time = 6 * 60
        end_time = 11 * 60
    else:
        start_time = 15 * 60
        end_time = 20 * 60

    # Copy the event List so the original scenario event queues are
    # not modified during the simulation run.
    simulation_events = events.copy()
    heapq.heapify(simulation_events)

    # Initialize the simulation state for queues, buses, and histories.
    stop_queues = {
        stop: []
        for stop in selected_stops
    }

    bus_occupancy = {}
    waiting_records = []
    queue_history = []
    bus_history = []

    peak_queue = {
        stop: 0
        for stop in selected_stops
    }

    peak_queue_time = {
        stop: None
        for stop in selected_stops
    }

    simulation_clock = start_time
    period_end_queue = None

```

```

period_end_recorded = False

# Record the initial empty queue state at every modeled stop.
for stop in selected_stops:
    queue_history.append({
        "Time": simulation_clock,
        "Stop Sequence": stop,
        "Queue Length": 0,
        "Event": "simulation_start"
    })

# Process passenger and bus events in chronological order.
while simulation_events:
    event = heapq.heappop(simulation_events)

    (
        event_time,
        priority,
        event_type,
        stop_sequence,
        event_id
    ) = event

    # Capture the queue state at the official end of the study
    # period before processing later events.
    if (
        not period_end_recorded
        and event_time > end_time
    ):
        period_end_queue = {
            stop: len(queue)
            for stop, queue in stop_queues.items()
        }

        period_end_recorded = True

    simulation_clock = event_time

    # Apply the appropriate event-processing rule.
    if event_type == "passenger_arrival":
        handle_passenger_arrival(

```

```

        event_time,
        stop_sequence,
        event_id,
        stop_queues,
        queue_history,
        peak_queue,
        peak_queue_time
    )
else:
    handle_bus_arrival(
        event_time,
        stop_sequence,
        event_id,
        stop_queues,
        bus_occupancy,
        waiting_records,
        queue_history,
        bus_history,
        occupancy_lookup,
        service_alighting_lookup,
        capacity=bus_capacity
    )

# If no event occurred after the official period boundary, use the
# final queue state as the recorded period-end queue.
if not period_end_recorded:
    period_end_queue = {
        stop: len(queue)
        for stop, queue in stop_queues.items()
    }

# Passengers still waiting after the final eligible bus form the
# final backlog for the period.
final_backlog = {
    stop: len(queue)
    for stop, queue in stop_queues.items()
}

# Convert recorded simulation histories to DataFrames for metric
# calculation and validation.
waiting_df = pd.DataFrame(waiting_records)

```

```

bus_history_df = pd.DataFrame(bus_history)
queue_history_df = pd.DataFrame(queue_history)

completed_passengers = len(waiting_df)
total_final_backlog = sum(final_backlog.values())

# Verify that every generated passenger is either boarded or remains
# in the final queue.
passenger_conservation = (
    generated_passengers
    == completed_passengers + total_final_backlog
)

# Calculate waiting-time metrics from passengers who successfully board.
mean_wait = waiting_df["Waiting Time"].mean()
median_wait = waiting_df["Waiting Time"].median()
p95_wait = waiting_df["Waiting Time"].quantile(0.95)
max_wait = waiting_df["Waiting Time"].max()

min_wait = (
    waiting_df["Waiting Time"].min()
    if len(waiting_df)
    else np.nan
)

# Verify that no completed passenger has a negative waiting time.
waiting_time_valid = (
    bool(
        (
            waiting_df["Waiting Time"] >= 0
        ).all()
    )
    if len(waiting_df)
    else True
)

# Calculate bus-occupancy limits observed during the simulation.
max_occupancy = int(
    bus_history_df["Occupancy After"].max()
)

```

```

min_occupancy = int(
    min(
        bus_history_df["Occupancy After"].min(),
        bus_history_df["Occupancy Before"].min()
    )
)

# Verify that bus occupancy remains within the modeled capacity
# before and after every bus event.
occupancy_valid = bool(
    bus_history_df[
        "Occupancy After"
    ].between(
        0,
        bus_capacity
    ).all()
    and bus_history_df[
        "Occupancy Before"
    ].between(
        0,
        bus_capacity
    ).all()
)

# Identify the largest queue observed across the five modeled stops.
overall_peak_queue = max(
    peak_queue.values()
)

overall_peak_stop = max(
    peak_queue,
    key=peak_queue.get
)

overall_peak_time = peak_queue_time[
    overall_peak_stop
]

# Store the period-level metrics and validation results.
result = {
    "Replication": int(replication),

```

```
"Seed": int(seed),
"Period": period,
"Generated Passengers": int(
    generated_passengers
),
"Completed Passengers": int(
    completed_passengers
),
"Mean Completed Wait": float(
    mean_wait
),
"Median Completed Wait": float(
    median_wait
),
"P95 Completed Wait": float(
    p95_wait
),
"Maximum Completed Wait": float(
    max_wait
),
"Minimum Completed Wait": float(
    min_wait
),
"Peak Queue": int(
    overall_peak_queue
),
"Peak Queue Stop": int(
    overall_peak_stop
),
"Peak Queue Time": float(
    overall_peak_time
),
"Period-End Queue": int(
    sum(period_end_queue.values())
),
"Final Backlog": int(
    total_final_backlog
),
"Mean Bus Occupancy": float(
    bus_history_df[
        "Occupancy After"
```

```

        ].mean()
    ),
    "Maximum Bus Occupancy": max_occupancy,
    "Minimum Bus Occupancy": min_occupancy,
    "Passenger Conservation": bool(
        passenger_conservation
    ),
    "Waiting Time Valid": waiting_time_valid,
    "Bus Occupancy Valid": occupancy_valid
}

# Calculate the corresponding queue and waiting-time metrics
# separately for each modeled stop.
stop_records = []

for stop in selected_stops:
    stop_waits = (
        waiting_df.loc[
            waiting_df["Stop Sequence"] == stop,
            "Waiting Time"
        ]
        if len(waiting_df)
        else pd.Series(dtype=float)
    )

    stop_records.append({
        "Replication": int(replication),
        "Seed": int(seed),
        "Period": period,
        "Stop Sequence": stop,
        "Completed Passengers": int(
            len(stop_waits)
        ),
        "Mean Wait": (
            float(stop_waits.mean())
            if len(stop_waits)
            else np.nan
        ),
        "P95 Wait": (
            float(
                stop_waits.quantile(0.95)
            )
        )
    })

```

```

        )
        if len(stop_waits)
        else np.nan
    ),
    "Peak Queue": int(
        peak_queue[stop]
    ),
    "Peak Queue Time": (
        peak_queue_time[stop]
    ),
    "Period-End Queue": int(
        period_end_queue[stop]
    ),
    "Final Backlog": int(
        final_backlog[stop]
    )
})

stop_level_df = pd.DataFrame(
    stop_records
)

return (
    result,
    waiting_df,
    bus_history_df,
    queue_history_df,
    stop_level_df
)

```

10. Passenger Stream Generators

Passenger arrivals are generated independently within each stop-hour using a uniform distribution over the corresponding 60-minute interval. The stop-hour passenger counts remain fixed for a scenario; only the within-hour arrival times vary across replications.

Baseline and Reduced-Service use the same Baseline passenger-demand table and the same replication seed. Their passenger streams are therefore identical for a matched replication.

High-Demand preserves the complete Baseline passenger stream and generates only the additional passengers required by the +40% stop-hour demand condition. These additional passengers use a separate deterministic random sub-stream so generating them does not alter the Baseline arrivals.

The Combined scenario uses the same High-Demand passenger generator because its passenger-demand condition is identical to High-Demand.

```
In [24]: # Generate one reproducible passenger stream from a supplied stop-hour
# demand table.
def generate_scenario_passengers(
    demand_df,
    seed,
    replication,
    id_prefix="P",
    id_start=1,
    source_label=None
):
    """Generate a reproducible passenger stream from a supplied stop-hour demand table."""
    rng = np.random.default_rng(seed)
    records = []

    # Generate passenger arrival times independently within each
    # corresponding 60-minute study-hour interval.
    for _, row in demand_df.iterrows():
        start_minute = int(row["Hour"]) * 60
        end_minute = start_minute + 60

        arrival_times = rng.uniform(
            start_minute,
            end_minute,
            int(row["Passenger Count"])
        )

        # Sort arrivals within the stop-hour cell before adding them
        # to the complete passenger stream.
        for arrival_time in sorted(arrival_times):
            records.append({
                "Stop Sequence": int(
                    row["Stop Sequence"]
```

```

    ),
    "Stop Name": row["Stop Name"],
    "Hour": int(row["Hour"]),
    "Period": (
        "Morning"
        if row["Hour"] < 12
        else "Evening"
    ),
    "Arrival Minutes": float(
        arrival_time
    )
})

passengers = pd.DataFrame(records)

# Assign a unique passenger identifier within the replication.
passengers.insert(
    0,
    "Passenger ID",
    [
        f"R{replication:02d}_{id_prefix}{i:05d}"
        for i in range(
            id_start,
            id_start + len(passengers)
        )
    ]
)

# Record the passenger source when a scenario contains more than
# one independently generated component.
if source_label is not None:
    passengers["Passenger Source"] = source_label

return passengers

```

```

In [25]: # Build a High-Demand passenger stream by preserving the complete
# Baseline stream and generating only the additional passengers.
def generate_high_demand_passengers(
    baseline_demand_df,
    additional_demand_df,

```

```

seed,
replication
):
# Generate the Baseline portion using the same demand table and
# replication seed used by the Baseline scenario.
"""Create the High-Demand stream by preserving Baseline passengers and generating only the additional demand."""
baseline_passengers = generate_scenario_passengers(
    baseline_demand_df,
    seed=seed,
    replication=replication,
    id_prefix="P",
    id_start=1,
    source_label="Baseline"
)

# Convert the additional-passenger table into the same input
# structure expected by the general passenger generator.
extra_demand_df = additional_demand_df[
    [
        "Stop Sequence",
        "Stop Name",
        "Hour",
        "Additional Passenger Count"
    ]
].rename(
    columns={
        "Additional Passenger Count":
            "Passenger Count"
    }
)

# Generate only the additional passengers from a separate,
# deterministic random sub-stream.
extra_passengers = generate_scenario_passengers(
    extra_demand_df,
    seed=(seed, 1),
    replication=replication,
    id_prefix="H",
    id_start=1,
    source_label="High-Demand Additional"
)

```

```

# Preserve the complete Baseline stream in its original order and
# append only the additional High-Demand passengers.
return pd.concat(
    [
        baseline_passengers,
        extra_passengers
    ],
    ignore_index=True
)

```

```

In [26]: # Verify that the Baseline portion of a High-Demand replication is
# unchanged when both are generated with the same replication seed.
check_seed = 999999
check_replication = 1

high_demand_check = generate_high_demand_passengers(
    baseline_demand,
    additional_demand,
    check_seed,
    check_replication
)

baseline_check = generate_scenario_passengers(
    baseline_demand,
    check_seed,
    check_replication,
    source_label="Baseline"
)

baseline_portion = (
    high_demand_check[
        high_demand_check["Passenger Source"]
        == "Baseline"
    ]
    .reset_index(drop=True)
)

baseline_stream_matches = np.allclose(
    baseline_portion["Arrival Minutes"].values,

```

```

    baseline_check["Arrival Minutes"].values
)

expected_high_demand_count = (
    len(baseline_check)
    + additional_passenger_total
)

high_demand_count_matches = (
    len(high_demand_check)
    == expected_high_demand_count
)

assert baseline_stream_matches, (
    "The Baseline portion of the High-Demand stream changed."
)

assert high_demand_count_matches, (
    "The High-Demand passenger stream has an unexpected passenger count."
)

print(
    "Baseline portion matches standalone Baseline generation:",
    baseline_stream_matches
)

print(
    "High-Demand passenger count:",
    len(high_demand_check),
    "=",
    len(baseline_check),
    "+",
    additional_passenger_total
)

```

Baseline portion matches standalone Baseline generation: True
 High-Demand passenger count: 1083 = 776 + 307

```

In [27]: # Provide scenario-specific passenger generators to the shared
         # replication runner.
         def baseline_generator(seed, replication):

```

```

    """Generate the Baseline passenger stream for one replication."""
    return generate_scenario_passengers(
        baseline_demand,
        seed,
        replication,
        id_prefix="P",
        id_start=1,
        source_label="Baseline"
    )

def high_demand_generator(seed, replication):
    """Generate the High-Demand passenger stream for one replication."""
    return generate_high_demand_passengers(
        baseline_demand,
        additional_demand,
        seed,
        replication
    )

def reduced_service_generator(seed, replication):
    """Generate Baseline-demand passengers for a Reduced-Service replication."""
    return generate_scenario_passengers(
        baseline_demand,
        seed,
        replication,
        id_prefix="P",
        id_start=1,
        source_label="Baseline"
    )

```

11. Reproducible Replication Seeds

All scenarios use matched replication seeds generated from the project master seed, 2026 .

A pool of 50 deterministic seeds is created so the simulation can begin with 30 matched replications and, if necessary, extend to 40 or 50 without changing any previously generated replication.

The first 30 generated seeds are checked against the saved Baseline replication-seed file. This ensures that scenario comparisons use the same stochastic passenger-arrival conditions as the established Baseline experiment.

```
In [28]: # Generate a deterministic pool of replication seeds from the project
# master seed.
seed_sequence = np.random.SeedSequence(
    master_seed
)

child_sequences = seed_sequence.spawn(
    precision_max_replications
)

seed_pool = [
    int(child.generate_state(1)[0])
    for child in child_sequences
]

seed_pool_df = pd.DataFrame({
    "Replication": range(
        1,
        precision_max_replications + 1
    ),
    "Seed": seed_pool
})

# Load the saved Baseline seeds used for the original 30-replication
# experiment.
official_baseline_seeds = pd.read_csv(
    processed_dir / "M66_Replication_Seeds.csv"
)

# Verify that the first 30 scenario seeds match the established
# Baseline replication seeds exactly.
baseline_seed_count = len(
    official_baseline_seeds
)

seeds_match_baseline = np.array_equal(
    seed_pool_df.loc[
```

```

        :baseline_seed_count - 1,
        "Seed"
    ].values,
    official_baseline_seeds["Seed"].values
)

assert seeds_match_baseline, (
    "Scenario replication seeds do not match the saved "
    "Baseline seeds."
)

print(
    f"First {baseline_seed_count} scenario seeds "
    "match the Baseline seed file:",
    seeds_match_baseline
)

seed_pool_df.head(10)

```

First 30 scenario seeds match the Baseline seed file: True

Out[28]:

	Replication	Seed
0	1	479243620
1	2	454024514
2	3	1818751028
3	4	2288458700
4	5	2115331525
5	6	2666256175
6	7	975381835
7	8	2535166222
8	9	1079577930
9	10	3715024065

12. Replication Precision Check

Simulation precision is evaluated using a 95% Student's t confidence interval across independent replications.

Precision is checked separately for the Morning and Evening periods using two primary simulation measures:

- Mean Completed Wait
- Peak Queue

For a metric with a replication mean below 10, the required confidence-interval half-width is at most 1.0. For a mean of 10 or greater, the required half-width is at most 10% of the replication mean.

Each scenario begins with 30 replications. If any required precision check fails, additional replications are added in batches of 10 up to a maximum of 50.

```
In [29]: # Evaluate the required 95% confidence-interval precision for the
# primary simulation metrics in both study periods.
def compute_precision_check(results_df):
    """Evaluate the project 95% confidence-interval precision rule for the primary metrics."""
    precision_records = []
    all_passed = True

    for period in ["Morning", "Evening"]:
        period_results = results_df[
            results_df["Period"] == period
        ]

        for metric in [
            "Mean Completed Wait",
            "Peak Queue"
        ]:
            values = period_results[metric]

            n = len(values)

            if n < 2:
                raise ValueError(
```

```

        "At least two replications are required "
        "for the precision check."
    )

    mean_value = values.mean()
    std_value = values.std(ddof=1)
    standard_error = (
        std_value / np.sqrt(n)
    )

    # Calculate the two-sided Student's t confidence interval.
    t_critical = t.ppf(
        precision_confidence_level
        + (
            1
            - precision_confidence_level
        ) / 2,
        df=n - 1
    )

    half_width = (
        t_critical
        * standard_error
    )

    # Apply the project's absolute or relative precision
    # threshold according to the replication mean.
    if mean_value < 10:
        required_half_width = 1.0
    else:
        required_half_width = (
            0.10 * mean_value
        )

    passed = (
        half_width
        <= required_half_width
    )

    all_passed = (
        all_passed

```

```

        and passed
    )

    precision_records.append({
        "Period": period,
        "Metric": metric,
        "Replications": n,
        "Mean": mean_value,
        "Standard Deviation": std_value,
        "CI Lower": (
            mean_value - half_width
        ),
        "CI Upper": (
            mean_value + half_width
        ),
        "Half-Width": half_width,
        "Required Half-Width":
            required_half_width,
        "Precision Passed": passed
    })

    return (
        pd.DataFrame(
            precision_records
        ),
        all_passed
    )

```

13. Scenario Replication Runner

The shared replication runner executes a complete scenario using its passenger generator, service schedule, starting-occupancy inputs, and alighting assignments.

Each scenario starts with 30 matched replications. After the initial batch, the precision check is applied. If the required precision is not achieved for Mean Completed Wait or Peak Queue in either study period, 10 additional replications are executed. This continues up to the 50-replication limit.

Previously completed replications are retained when the experiment is extended. Only the additional required replications are simulated.

The same runner is used for Baseline, High-Demand, Reduced-Service, and Combined conditions so scenario comparisons use the same simulation and evaluation process.

```
In [30]: # Run a complete scenario with matched replication seeds and extend the
# experiment only when the required precision has not yet been achieved.
def run_scenario_replications(
    scenario_name,
    passenger_generator_fn,
    service_stop_times,
    trip_period_lookup_table,
    occupancy_lookup_table,
    alighting_lookup_table,
    keep_passenger_streams=False
):
    """Run matched scenario replications and extend the experiment only when the required precision is not met."""
    results = []
    stop_level_records = []
    passenger_stream_frames = []

    current_target = (
        precision_start_replications
    )

    completed_reps = 0

    while True:
        # Run only the replications that have not already been completed.
        for replication in range(
            completed_reps + 1,
            current_target + 1
        ):
            seed = int(
                seed_pool_df.loc[
                    seed_pool_df["Replication"]
                    == replication,
                    "Seed"
                ].iloc[0]
            )
```

```

passengers = (
    passenger_generator_fn(
        seed,
        replication
    )
)

# Retain a Labeled copy of the passenger stream when it is
# required as an input to downstream optimization.
if keep_passenger_streams:
    passenger_stream = passengers.copy()

    passenger_stream.insert(
        0,
        "Replication",
        replication
    )

    passenger_stream.insert(
        1,
        "Seed",
        seed
    )

    passenger_stream_frames.append(
        passenger_stream
    )

# Build the Morning and Evening event queues using the
# scenario-specific passenger and service inputs.
(
    morning_events,
    evening_events
) = create_period_event_queues(
    passengers,
    service_stop_times,
    trip_period_lookup_table
)

morning_n = int(
    (

```

```

        passengers["Period"]
        == "Morning"
    ).sum()
)

evening_n = int(
    (
        passengers["Period"]
        == "Evening"
    ).sum()
)

# Run the two study periods independently using the same
# replication number and random seed.
(
    morning_result,
    —,
    —,
    —,
    morning_stop
) = run_service_simulation(
    "Morning",
    morning_events,
    morning_n,
    replication,
    seed,
    occupancy_lookup_table,
    alighting_lookup_table
)

(
    evening_result,
    —,
    —,
    —,
    evening_stop
) = run_service_simulation(
    "Evening",
    evening_events,
    evening_n,
    replication,

```

```

        seed,
        occupancy_lookup_table,
        alighting_lookup_table
    )

    results.extend([
        morning_result,
        evening_result
    ])

    stop_level_records.extend([
        morning_stop,
        evening_stop
    ])

    completed_reps = current_target

    # Check whether the completed replication set meets the
    # predefined precision requirement.
    results_df = pd.DataFrame(
        results
    )

    (
        precision_df,
        all_passed
    ) = compute_precision_check(
        results_df
    )

    # Stop when precision is achieved or when the maximum
    # replication limit has been reached.
    if (
        all_passed
        or completed_reps
        >= precision_max_replications
    ):
        break

    current_target = min(
        completed_reps

```

```

        + precision_increment,
        precision_max_replications
    )

    # Add the scenario identifier to the complete result tables.
    results_df = pd.DataFrame(
        results
    )

    results_df.insert(
        0,
        "Scenario",
        scenario_name
    )

    stop_level_df = pd.concat(
        stop_level_records,
        ignore_index=True
    )

    stop_level_df.insert(
        0,
        "Scenario",
        scenario_name
    )

    # Recalculate the final precision table using all replications
    # retained for the completed scenario.
    (
        precision_df,
        all_passed
    ) = compute_precision_check(
        results_df
    )

    precision_df.insert(
        0,
        "Scenario",
        scenario_name
    )

```

```

# Combine retained passenger streams when requested.
passenger_streams_df = (
    pd.concat(
        passenger_stream_frames,
        ignore_index=True
    )
    if keep_passenger_streams
    else None
)

print(
    f"[{scenario_name}] "
    f"Replications used: {completed_reps} "
    f"(started at "
    f"{precision_start_replications}, "
    f"cap {precision_max_replications}) "
    f"| Precision requirement met: "
    f"{all_passed}"
)

return (
    results_df,
    stop_level_df,
    precision_df,
    passenger_streams_df,
    completed_reps
)

```

14. Run High-Demand Scenario

The High-Demand scenario uses the +40% selected-stop passenger stream together with the unchanged Baseline GTFS service, starting occupancy, and alighting assignments.

The scenario begins with 30 matched replications and may extend to a maximum of 50 if the predefined precision requirement is not satisfied.

```

In [31]: # Run High-Demand using the increased selected-stop passenger demand
# together with the unchanged Baseline service inputs.
(

```

```

    high_demand_results_df,
    high_demand_stop_df,
    high_demand_precision_df,
    high_demand_master_streams,
    high_demand_replications_used
) = run_scenario_replications(
    scenario_name="High-Demand",
    passenger_generator_fn=high_demand_generator,
    service_stop_times=study_stop_times,
    trip_period_lookup_table=trip_period_lookup,
    occupancy_lookup_table=initial_occupancy_lookup,
    alighting_lookup_table=alighting_lookup,
    keep_passenger_streams=True
)

# Fail immediately if a core simulation validity condition is violated.
assert high_demand_results_df[
    "Passenger Conservation"
].all(), (
    "High-Demand failed passenger-conservation validation."
)

assert high_demand_results_df[
    "Waiting Time Valid"
].all(), (
    "High-Demand produced a negative completed waiting time."
)

assert high_demand_results_df[
    "Bus Occupancy Valid"
].all(), (
    "High-Demand produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    high_demand_results_df[
        "Passenger Conservation"
    ].all()
)
print(

```

```

    "Waiting times valid:",
    high_demand_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    high_demand_results_df[
        "Bus Occupancy Valid"
    ].all()
)

# Verify that the retained High-Demand passenger streams contain the
# replication labels required by the downstream GA notebook.
required_master_stream_columns = {
    "Replication",
    "Seed",
    "Passenger ID",
    "Stop Sequence",
    "Hour",
    "Period",
    "Arrival Minutes"
}

master_stream_columns_ok = (
    required_master_stream_columns
    .issubset(
        high_demand_master_streams.columns
    )
)

master_stream_replications_ok = (
    high_demand_master_streams[
        "Replication"
    ].nunique()
    == high_demand_replications_used
)

assert master_stream_columns_ok, (
    "High-Demand master passenger streams are missing "

```

```

        "columns required by the GA notebook."
    )

    assert master_stream_replications_ok, (
        "High-Demand master passenger streams do not contain "
        "all retained replications."
    )

    print(
        "High-Demand master streams ready for GA:",
        master_stream_columns_ok
        and master_stream_replications_ok
    )

    print(
        "High-Demand master-stream replications:",
        high_demand_master_streams[
            "Replication"
        ].unique()
    )

    high_demand_precision_df.round(3)

```

```

[High-Demand] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True
Passenger conservation: True
Waiting times valid: True
Bus occupancy valid: True
High-Demand master streams ready for GA: True
High-Demand master-stream replications: 30

```

Out[31]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half-Width	Required Half-Width	Precision Passed
0	High-Demand	Morning	Mean Completed Wait	30	3.663	0.076	3.634	3.691	0.028	1.000	True
1	High-Demand	Morning	Peak Queue	30	23.367	2.684	22.364	24.369	1.002	2.337	True
2	High-Demand	Evening	Mean Completed Wait	30	3.641	0.131	3.592	3.690	0.049	1.000	True
3	High-Demand	Evening	Peak Queue	30	13.900	2.074	13.126	14.674	0.774	1.390	True

15. Run Reduced-Service Scenario

The Reduced-Service scenario keeps Baseline passenger demand unchanged while using the Reduced-Service reference schedule.

Because fewer buses serve each study hour, the recalculated Reduced-Service starting occupancy and alighting assignments are supplied to the same simulation engine.

The scenario begins with 30 matched replications and may extend to a maximum of 50 if required by the precision rule.

```
In [32]: # Run Reduced-Service using the unchanged Baseline passenger stream
# and the reduced reference service inputs.
(
    reduced_service_results_df,
    reduced_service_stop_df,
    reduced_service_precision_df,
    _,
    reduced_service_replications_used
) = run_scenario_replications(
    scenario_name="Reduced-Service",
    passenger_generator_fn=reduced_service_generator,
    service_stop_times=reduced_study_stop_times,
    trip_period_lookup_table=reduced_trip_period_lookup,
    occupancy_lookup_table=reduced_initial_occupancy_lookup,
```

```

    alighting_lookup_table=reduced_alighting_lookup,
    keep_passenger_streams=False
)

# Fail immediately if a core simulation validity condition is violated.
assert reduced_service_results_df[
    "Passenger Conservation"
].all(), (
    "Reduced-Service failed passenger-conservation validation."
)

assert reduced_service_results_df[
    "Waiting Time Valid"
].all(), (
    "Reduced-Service produced a negative completed waiting time."
)

assert reduced_service_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Reduced-Service produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    reduced_service_results_df[
        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    reduced_service_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    reduced_service_results_df[
        "Bus Occupancy Valid"
    ].all()
)

```

```
reduced_service_precision_df.round(3)
```

[Reduced-Service] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True

Passenger conservation: True

Waiting times valid: True

Bus occupancy valid: True

Out[32]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half-Width	Required Half-Width	Precision Passed
0	Reduced-Service	Morning	Mean Completed Wait	30	4.850	0.166	4.788	4.912	0.062	1.000	True
1	Reduced-Service	Morning	Peak Queue	30	24.500	3.026	23.370	25.630	1.130	2.450	True
2	Reduced-Service	Evening	Mean Completed Wait	30	4.368	0.230	4.283	4.454	0.086	1.000	True
3	Reduced-Service	Evening	Peak Queue	30	10.267	1.437	9.730	10.803	0.537	1.027	True

16. Run Combined Scenario

The Combined scenario applies both stress conditions at the same time:

- boarding demand at the five modeled stops is increased by 40%; and
- one published reference trip is removed from every study hour.

The passenger stream is therefore identical to the High-Demand condition, while the service schedule, starting occupancy, and alighting assignments are identical to the Reduced-Service condition.

The upstream occupancy and alighting inputs are not multiplied by 1.40 because the demand increase is defined only for boarding demand at the five modeled stops.

The same simulation engine and matched replication seeds are used as in all other scenarios.

```

In [33]: # Run Combined using the High-Demand passenger stream together with
# the complete Reduced-Service service-side configuration.
(
    combined_results_df,
    combined_stop_df,
    combined_precision_df,
    _,
    combined_replications_used
) = run_scenario_replications(
    scenario_name="Combined",
    passenger_generator_fn=high_demand_generator,
    service_stop_times=reduced_study_stop_times,
    trip_period_lookup_table=reduced_trip_period_lookup,
    occupancy_lookup_table=reduced_initial_occupancy_lookup,
    alighting_lookup_table=reduced_alighting_lookup,
    keep_passenger_streams=False
)

# Fail immediately if a core simulation validity condition is violated.
assert combined_results_df[
    "Passenger Conservation"
].all(), (
    "Combined failed passenger-conservation validation."
)

assert combined_results_df[
    "Waiting Time Valid"
].all(), (
    "Combined produced a negative completed waiting time."
)

assert combined_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Combined produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    combined_results_df[

```

```

        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    combined_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    combined_results_df[
        "Bus Occupancy Valid"
    ].all()
)

combined_precision_df.round(3)

```

[Combined] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True

Passenger conservation: True

Waiting times valid: True

Bus occupancy valid: True

Out[33]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half- Width	Required Half- Width	Precision Passed
0	Combined	Morning	Mean Completed Wait	30	4.817	0.153	4.760	4.874	0.057	1.00	True
1	Combined	Morning	Peak Queue	30	32.900	4.302	31.294	34.506	1.606	3.29	True
2	Combined	Evening	Mean Completed Wait	30	4.331	0.176	4.265	4.397	0.066	1.00	True
3	Combined	Evening	Peak Queue	30	14.400	1.812	13.723	15.077	0.677	1.44	True

17. Run Baseline Reference for Comparison

The Baseline condition is rerun through the same shared scenario runner used by the three stress scenarios.

This provides a directly comparable reference using the same replication-seed pool, simulation engine, precision rule, service inputs, and output structure used in the current scenario experiment.

```
In [34]: # Rerun Baseline through the same scenario runner used for all
# stress conditions so the resulting tables have a common structure.
(
    baseline_results_df,
    baseline_stop_df,
    baseline_precision_df,
    _,
    baseline_replications_used
) = run_scenario_replications(
    scenario_name="Baseline",
    passenger_generator_fn=baseline_generator,
    service_stop_times=study_stop_times,
    trip_period_lookup_table=trip_period_lookup,
    occupancy_lookup_table=initial_occupancy_lookup,
    alighting_lookup_table=alighting_lookup,
    keep_passenger_streams=False
)

# Fail immediately if a core simulation validity condition is violated.
assert baseline_results_df[
    "Passenger Conservation"
].all(), (
    "Baseline failed passenger-conservation validation."
)

assert baseline_results_df[
    "Waiting Time Valid"
].all(), (
    "Baseline produced a negative completed waiting time."
)

assert baseline_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Baseline produced a bus occupancy outside the valid range."
)
```

```

print(
    "Passenger conservation:",
    baseline_results_df[
        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    baseline_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    baseline_results_df[
        "Bus Occupancy Valid"
    ].all()
)

baseline_precision_df.round(3)

```

[Baseline] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True

Passenger conservation: True

Waiting times valid: True

Bus occupancy valid: True

Out[34]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half-Width	Required Half-Width	Precision Passed
0	Baseline	Morning	Mean Completed Wait	30	3.657	0.110	3.616	3.698	0.041	1.000	True
1	Baseline	Morning	Peak Queue	30	17.267	1.999	16.520	18.013	0.746	1.727	True
2	Baseline	Evening	Mean Completed Wait	30	3.651	0.164	3.590	3.712	0.061	1.000	True
3	Baseline	Evening	Peak Queue	30	9.867	1.570	9.280	10.453	0.586	1.000	True

18. Validate Simulation Results

Validation is consolidated across Baseline, High-Demand, Reduced-Service, and Combined before the scenario results are summarized or interpreted.

For each scenario, the checks confirm that:

- every generated passenger is accounted for as either successfully boarded or remaining in final backlog;
- no completed passenger has a negative waiting time;
- bus occupancy remains between zero and the modeled 60-passenger capacity;
- the required 95% confidence-interval precision is satisfied; and
- every executed replication uses the corresponding deterministic seed from the common project seed pool.

The maximum observed bus occupancy is also reported so capacity behavior can be inspected directly rather than relying only on a Boolean validity flag.

The notebook stops if any required validation condition fails.

```
In [35]: # Collect the result and precision tables for all four simulation
# conditions in one structure for consolidated validation.
scenario_validation_inputs = {
    "Baseline": {
        "results": baseline_results_df,
        "precision": baseline_precision_df
    },
    "High-Demand": {
        "results": high_demand_results_df,
        "precision": high_demand_precision_df
    },
    "Reduced-Service": {
        "results": reduced_service_results_df,
        "precision": reduced_service_precision_df
    },
    "Combined": {
        "results": combined_results_df,
        "precision": combined_precision_df
    }
}
```

```

validation_records = []

# Evaluate the same core simulation and precision checks for every
# scenario using the complete set of retained replications.
for scenario_name, scenario_data in scenario_validation_inputs.items():
    results_df = scenario_data["results"]
    precision_df = scenario_data["precision"]

    replications_used = int(
        results_df["Replication"].nunique()
    )

    passenger_conservation_ok = bool(
        results_df["Passenger Conservation"].all()
    )

    waiting_time_ok = bool(
        results_df["Waiting Time Valid"].all()
    )

    bus_occupancy_ok = bool(
        results_df["Bus Occupancy Valid"].all()
    )

    precision_ok = bool(
        precision_df["Precision Passed"].all()
    )

    max_observed_occupancy = int(
        results_df["Maximum Bus Occupancy"].max()
    )

    validation_records.append({
        "Scenario": scenario_name,
        "Replications Used": replications_used,
        "Passenger Conservation": passenger_conservation_ok,
        "Waiting Time Valid": waiting_time_ok,
        "Bus Occupancy Valid": bus_occupancy_ok,
        "Precision Passed": precision_ok,
        "Maximum Observed Bus Occupancy": max_observed_occupancy,
        "Modeled Bus Capacity": bus_capacity
    })

```

```

    })

validation_summary_df = pd.DataFrame(
    validation_records
)

# Mark a scenario as fully valid only when every required validation
# and precision condition passes.
validation_summary_df["All Required Checks Passed"] = (
    validation_summary_df[
        [
            "Passenger Conservation",
            "Waiting Time Valid",
            "Bus Occupancy Valid",
            "Precision Passed"
        ]
    ]
    .all(axis=1)
)

assert validation_summary_df[
    "All Required Checks Passed"
].all(), (
    "At least one scenario failed a required simulation "
    "or precision validation check."
)

print(
    "All required validation checks passed:",
    validation_summary_df[
        "All Required Checks Passed"
    ].all()
)

validation_summary_df

```

All required validation checks passed: True

Out[35]:

	Scenario	Replications Used	Passenger Conservation	Waiting Time Valid	Bus Occupancy Valid	Precision Passed	Maximum Observed Bus Occupancy	Modeled Bus Capacity	All Required Checks Passed
0	Baseline	30	True	True	True	True	33	60	True
1	High-Demand	30	True	True	True	True	45	60	True
2	Reduced-Service	30	True	True	True	True	52	60	True
3	Combined	30	True	True	True	True	60	60	True

```
In [36]: # Verify that every executed replication uses the deterministic seed
# assigned to that replication in the common project seed pool.
matched_seed_records = []

for scenario_name, scenario_data in scenario_validation_inputs.items():
    scenario_seed_map = (
        scenario_data["results"][
            ["Replication", "Seed"]
        ]
        .drop_duplicates()
        .sort_values("Replication")
        .reset_index(drop=True)
    )

    expected_seed_map = (
        seed_pool_df[
            seed_pool_df["Replication"].isin(
                scenario_seed_map["Replication"]
            )
        ]
        .sort_values("Replication")
        .reset_index(drop=True)
        .rename(
            columns={
                "Seed": "Expected Seed"
            }
        )
    )
```

```

    )
)

seed_check = scenario_seed_map.merge(
    expected_seed_map,
    on="Replication",
    how="left"
)

seeds_match = bool(
    seed_check["Expected Seed"].notna().all()
    and (
        seed_check["Seed"]
        == seed_check["Expected Seed"]
    ).all()
)

matched_seed_records.append({
    "Scenario": scenario_name,
    "Replications Used": len(
        scenario_seed_map
    ),
    "Matched Common Seed Pool": seeds_match
})

matched_seed_validation_df = pd.DataFrame(
    matched_seed_records
)

assert matched_seed_validation_df[
    "Matched Common Seed Pool"
].all(), (
    "At least one scenario does not use the expected "
    "deterministic replication seeds."
)

print(
    "All scenarios use the common replication seed pool:",
    matched_seed_validation_df[
        "Matched Common Seed Pool"
    ].all()
)

```

```
)  
  
matched_seed_validation_df
```

All scenarios use the common replication seed pool: True

Out[36]:

	Scenario	Replications Used	Matched Common Seed Pool
0	Baseline	30	True
1	High-Demand	30	True
2	Reduced-Service	30	True
3	Combined	30	True

```
In [37]: # Calculate the total generated passenger count within each complete  
# replication so the executed scenarios can be checked against their  
# intended demand conditions.  
scenario_passenger_records = []  
  
for scenario_name, scenario_data in scenario_validation_inputs.items():  
    results_df = scenario_data["results"]  
  
    generated_per_replication = (  
        results_df  
        .groupby("Replication") [  
            "Generated Passengers"  
        ]  
        .sum()  
    )  
  
    consistent_count = (  
        generated_per_replication.nunique()  
        == 1  
    )  
  
    passenger_count = int(  
        generated_per_replication.iloc[0]  
    )
```

```

scenario_passenger_records.append({
    "Scenario": scenario_name,
    "Passengers per Replication": passenger_count,
    "Consistent Across Replications": consistent_count
})

scenario_passenger_validation_df = pd.DataFrame(
    scenario_passenger_records
)

# Check the expected passenger-demand relationship between scenarios.
passenger_count_lookup = (
    scenario_passenger_validation_df
    .set_index("Scenario")[
        "Passengers per Replication"
    ]
    .to_dict()
)

demand_definition_ok = (
    passenger_count_lookup["Baseline"]
    == baseline_total
    and passenger_count_lookup["Reduced-Service"]
    == baseline_total
    and passenger_count_lookup["High-Demand"]
    == high_demand_total
    and passenger_count_lookup["Combined"]
    == high_demand_total
    and scenario_passenger_validation_df[
        "Consistent Across Replications"
    ].all()
)

assert demand_definition_ok, (
    "Executed scenario passenger counts do not match "
    "the defined demand conditions."
)

print(
    "Scenario passenger-demand definitions preserved:",
    demand_definition_ok
)

```

```
)  
  
scenario_passenger_validation_df
```

Scenario passenger-demand definitions preserved: True

Out[37]:

	Scenario	Passengers per Replication	Consistent Across Replications
0	Baseline	776	True
1	High-Demand	1083	True
2	Reduced-Service	776	True
3	Combined	1083	True

19. Prepare Genetic Algorithm Inputs

The same candidate-service construction is used for every operating condition. The scenario context changes only the passenger streams, fixed service budget, hourly feasibility bounds, reference plan, and reference backlog thresholds.

Baseline and High-Demand use the original 43-trip Morning and 44-trip Evening budgets with 5-to-12 hourly bounds. Reduced-Service and Combined use the reduced 38-trip Morning and 39-trip Evening budgets with 4-to-12 hourly bounds.

Candidate starting occupancy is recomputed for each candidate hour from the representative upstream net flow divided by the candidate number of buses in that hour. Alighting targets remain unchanged and are redistributed across the candidate buses serving each stop-hour.

```
In [38]: # Define the GA search settings used consistently across all scenarios.  
optimization_replication_count = 5  
optimizer_seeds = [101, 202, 303]  
evaluation_budget = 10  
ga_population_size = 6  
  
# Baseline and High-Demand use the observed GTFS hourly range.  
baseline_min_hourly_trips = int(  
    baseline_service_plan["Trips"].min()  
)  
baseline_max_hourly_trips = int(
```

```

        baseline_service_plan["Trips"].max()
    )

    # Reduced-Service and Combined use the fixed reduced budgets with
    # the final 4-to-12 hourly feasibility range.
    reduced_min_hourly_trips = 4
    reduced_max_hourly_trips = 12

    reduced_service_budget = (
        reduced_service_plan
        .groupby("Period")["Trips"]
        .sum()
        .to_dict()
    )

    baseline_plan_lookup = {
        period: tuple(
            baseline_service_plan.loc[
                baseline_service_plan["Period"] == period
            ]
            .sort_values("Hour")["Trips"]
            .astype(int)
            .tolist()
        )
        for period in ["Morning", "Evening"]
    }

    reduced_reference_plan_lookup = {
        period: tuple(
            reduced_service_plan.loc[
                reduced_service_plan["Period"] == period
            ]
            .sort_values("Hour")["Trips"]
            .astype(int)
            .tolist()
        )
        for period in ["Morning", "Evening"]
    }

    assert service_budget == {
        "Morning": 43,

```

```

    "Evening": 44
}, "Baseline service budgets do not match the verified GTFS reference."

assert reduced_service_budget == {
    "Morning": 38,
    "Evening": 39
}, "Reduced-Service budgets do not match the defined scenario."

assert all(
    reduced_min_hourly_trips <= trips <= reduced_max_hourly_trips
    for plan in reduced_reference_plan_lookup.values()
    for trips in plan
), "Reduced-Service reference plan falls outside the 4-to-12 bounds."

print("Baseline/High-Demand bounds:",
      baseline_min_hourly_trips, "to", baseline_max_hourly_trips)
print("Reduced-Service/Combined bounds:",
      reduced_min_hourly_trips, "to", reduced_max_hourly_trips)
print("Baseline budgets:", service_budget)
print("Reduced budgets:", reduced_service_budget)

```

Baseline/High-Demand bounds: 5 to 12

Reduced-Service/Combined bounds: 4 to 12

Baseline budgets: {'Evening': 44, 'Morning': 43}

Reduced budgets: {'Evening': 39, 'Morning': 38}

```

In [39]: # Measure the median travel time from Stop 6 to each modeled stop
# for every study hour using the verified baseline GTFS service.
trip_times = study_stop_times[
    ["trip_id", "stop_sequence", "arrival_minutes"]
].merge(
    trip_initial_occupancy[
        ["trip_id", "Hour"]
    ],
    on="trip_id",
    how="left"
)

stop6_times = (
    trip_times.loc[
        trip_times["stop_sequence"] == 6,

```

```

        ["trip_id", "arrival_minutes"]
    ]
    .rename(
        columns={
            "arrival_minutes": "stop6_arrival"
        }
    )
)

trip_times = trip_times.merge(
    stop6_times,
    on="trip_id",
    how="left"
)

trip_times["Travel Offset"] = (
    trip_times["arrival_minutes"]
    - trip_times["stop6_arrival"]
)

hourly_travel_offsets = (
    trip_times
    .groupby(
        ["Hour", "stop_sequence"]
    )["Travel Offset"]
    .median()
    .reset_index()
    .rename(
        columns={
            "stop_sequence": "Stop Sequence",
            "Travel Offset": "Median Offset Minutes"
        }
    )
)

travel_offset_lookup = (
    hourly_travel_offsets
    .set_index(
        ["Hour", "Stop Sequence"]
    )["Median Offset Minutes"]
    .to_dict()
)

```

```

)

required_offset_keys = {
    (hour, stop)
    for hour in study_hours
    for stop in selected_stops
}

assert required_offset_keys.issubset(
    travel_offset_lookup.keys()
), "At least one hour-stop travel offset is missing."

print(
    "Hour-stop travel offsets available:",
    len(travel_offset_lookup)
)

```

Hour-stop travel offsets available: 50

```

In [40]: # Build deterministic passenger-stream dictionaries for any required
# replication set using the same generators and seed pool as the reference runs.
seed_lookup = (
    seed_pool_df
    .set_index("Replication")["Seed"]
    .to_dict()
)

def build_passenger_streams(
    passenger_generator_fn,
    replication_ids
):
    """Build a replication-indexed dictionary of reproducible passenger streams for GA evaluation."""
    # Store one reproducible passenger table per replication for repeated GA evaluation.
    streams = {}

    # Rebuild each stream from its recorded replication seed.
    for replication in replication_ids:
        replication = int(replication)
        seed = int(
            seed_lookup[replication]
        )

```

```

        streams[replication] = (
            passenger_generator_fn(
                seed,
                replication
            )
        )

    return streams

def scenario_replication_ids(
    results_df
):
    """Return the sorted replication identifiers available in a scenario result table."""
    return sorted(
        results_df["Replication"]
        .astype(int)
        .unique()
        .tolist()
    )

baseline_validation_ids = (
    scenario_replication_ids(
        baseline_results_df
    )
)

high_demand_validation_ids = (
    scenario_replication_ids(
        high_demand_results_df
    )
)

reduced_service_validation_ids = (
    scenario_replication_ids(
        reduced_service_results_df
    )
)

```

```

combined_validation_ids = (
    scenario_replication_ids(
        combined_results_df
    )
)

baseline_passengers_by_replication = (
    build_passenger_streams(
        baseline_generator,
        baseline_validation_ids
    )
)

high_demand_ga_replication_ids = sorted(
    set(high_demand_validation_ids)
    | set(combined_validation_ids)
)

high_demand_passengers_by_replication = (
    build_passenger_streams(
        high_demand_generator,
        high_demand_ga_replication_ids
    )
)

reduced_service_passengers_by_replication = (
    build_passenger_streams(
        baseline_generator,
        reduced_service_validation_ids
    )
)

print(
    "Baseline GA passenger streams:",
    len(baseline_passengers_by_replication)
)

print(
    "High-Demand/Combined GA passenger streams:",
    len(high_demand_passengers_by_replication)
)

print(

```

```
    "Reduced-Service GA passenger streams:",
    len(reduced_service_passengers_by_replication)
)
```

Baseline GA passenger streams: 30

High-Demand/Combined GA passenger streams: 30

Reduced-Service GA passenger streams: 30

```
In [41]: # Package every scenario-varying input used by the optimizer.
def make_scenario_context(
    name,
    passengers_by_replication,
    service_budget,
    min_hourly_trips,
    max_hourly_trips,
    reference_plan_lookup,
    search_replication_ids,
    validation_replication_ids,
    upstream_flow_lookup,
    alighting_targets,
    travel_offset_lookup,
    feasibility_backlog
):
    """Package all scenario-varying service, demand, validation, and lookup inputs used by the generalized GA."""
    # Keep every scenario-specific input together so the generalized GA does not rely on hidden scenario state.
    return {
        "name": name,
        "passengers_by_replication":
            passengers_by_replication,
        "seed_lookup": dict(
            seed_lookup
        ),
        "service_budget": dict(
            service_budget
        ),
        "min_hourly_trips":
            int(min_hourly_trips),
        "max_hourly_trips":
            int(max_hourly_trips),
        "reference_plan_lookup":
            dict(reference_plan_lookup),
```

```

        "search_replication_ids":
            list(search_replication_ids),
        "validation_replication_ids":
            list(validation_replication_ids),
        "upstream_flow_lookup":
            dict(upstream_flow_lookup),
        "alighting_targets":
            alighting_targets.copy(),
        "travel_offset_lookup":
            dict(travel_offset_lookup),
        "feasibility_backlog":
            dict(feasibility_backlog),
        "fitness_cache": {}
    }
}

```

```

def build_backlog_reference(
    results_df,
    replication_ids
):
    """Calculate the reference mean final backlog by study period for a selected replication set."""
    # Use only the requested matched replications when computing the reference safeguard level.
    return (
        results_df.loc[
            results_df["Replication"].isin(
                replication_ids
            )
        ]
        .groupby("Period")[
            "Final Backlog"
        ]
        .mean()
        .to_dict()
    )

def first_search_replications(
    validation_ids
):
    """Select the fixed first matched replications used during the GA search stage."""
    # Fail early if the fixed GA search set cannot be formed from the available validation replications.

```

```

if len(validation_ids) < optimization_replication_count:
    raise ValueError(
        "At least five matched replications are required "
        "for the GA search."
    )

# Preserve the agreed first matched replications as the search subset.
return list(
    validation_ids[
        :optimization_replication_count
    ]
)

baseline_search_ids = (
    first_search_replications(
        baseline_validation_ids
    )
)

high_demand_search_ids = (
    first_search_replications(
        high_demand_validation_ids
    )
)

reduced_service_search_ids = (
    first_search_replications(
        reduced_service_validation_ids
    )
)

combined_search_ids = (
    first_search_replications(
        combined_validation_ids
    )
)

baseline_ctx = make_scenario_context(
    name="Baseline",
    passengers_by_replication=

```

```

        baseline_passengers_by_replication,
        service_budget=service_budget,
        min_hourly_trips=
            baseline_min_hourly_trips,
        max_hourly_trips=
            baseline_max_hourly_trips,
        reference_plan_lookup=
            baseline_plan_lookup,
        search_replication_ids=
            baseline_search_ids,
        validation_replication_ids=
            baseline_validation_ids,
        upstream_flow_lookup=
            upstream_net_flow_by_hour,
        alighting_targets=
            hourly_alighting_targets,
        travel_offset_lookup=
            travel_offset_lookup,
        feasibility_backlog=
            build_backlog_reference(
                baseline_results_df,
                baseline_search_ids
            )
    )
)

high_demand_ctx = make_scenario_context(
    name="High-Demand",
    passengers_by_replication=
        high_demand_passengers_by_replication,
    service_budget=service_budget,
    min_hourly_trips=
        baseline_min_hourly_trips,
    max_hourly_trips=
        baseline_max_hourly_trips,
    reference_plan_lookup=
        baseline_plan_lookup,
    search_replication_ids=
        high_demand_search_ids,
    validation_replication_ids=
        high_demand_validation_ids,
    upstream_flow_lookup=

```

```

        upstream_net_flow_by_hour,
    alighting_targets=
        hourly_alighting_targets,
    travel_offset_lookup=
        travel_offset_lookup,
    feasibility_backlog=
        build_backlog_reference(
            high_demand_results_df,
            high_demand_search_ids
        )
)

reduced_service_ctx = make_scenario_context(
    name="Reduced-Service",
    passengers_by_replication=
        reduced_service_passengers_by_replication,
    service_budget=
        reduced_service_budget,
    min_hourly_trips=
        reduced_min_hourly_trips,
    max_hourly_trips=
        reduced_max_hourly_trips,
    reference_plan_lookup=
        reduced_reference_plan_lookup,
    search_replication_ids=
        reduced_service_search_ids,
    validation_replication_ids=
        reduced_service_validation_ids,
    upstream_flow_lookup=
        upstream_net_flow_by_hour,
    alighting_targets=
        hourly_alighting_targets,
    travel_offset_lookup=
        travel_offset_lookup,
    feasibility_backlog=
        build_backlog_reference(
            reduced_service_results_df,
            reduced_service_search_ids
        )
)

```

```

combined_ctx = make_scenario_context(
    name="Combined",
    passengers_by_replication=
        high_demand_passengers_by_replication,
    service_budget=
        reduced_service_budget,
    min_hourly_trips=
        reduced_min_hourly_trips,
    max_hourly_trips=
        reduced_max_hourly_trips,
    reference_plan_lookup=
        reduced_reference_plan_lookup,
    search_replication_ids=
        combined_search_ids,
    validation_replication_ids=
        combined_validation_ids,
    upstream_flow_lookup=
        upstream_net_flow_by_hour,
    alighting_targets=
        hourly_alighting_targets,
    travel_offset_lookup=
        travel_offset_lookup,
    feasibility_backlog=
        build_backlog_reference(
            combined_results_df,
            combined_search_ids
        )
)

scenario_contexts = {
    "Baseline": baseline_ctx,
    "High-Demand": high_demand_ctx,
    "Reduced-Service": reduced_service_ctx,
    "Combined": combined_ctx
}

context_summary = pd.DataFrame([
    {
        "Scenario": name,
        "Morning Budget":
            ctx["service_budget"]["Morning"],
    }
])

```

```

    "Evening Budget":
        ctx["service_budget"]["Evening"],
    "Minimum Hourly Trips":
        ctx["min_hourly_trips"],
    "Maximum Hourly Trips":
        ctx["max_hourly_trips"],
    "Search Replications":
        len(ctx["search_replication_ids"]),
    "Validation Replications":
        len(ctx["validation_replication_ids"])
}
for name, ctx in scenario_contexts.items()
])

context_summary

```

Out[41]:

	Scenario	Morning Budget	Evening Budget	Minimum Hourly Trips	Maximum Hourly Trips	Search Replications	Validation Replications
0	Baseline	43	44	5	12	5	30
1	High-Demand	43	44	5	12	5	30
2	Reduced-Service	38	39	4	12	5	30
3	Combined	38	39	4	12	5	30

20. Candidate Service Evaluation and Genetic Algorithm

Each candidate vector contains five integer hourly trip counts for one study period. A candidate must remain within the scenario-specific hourly bounds and sum exactly to the scenario's fixed period budget.

The GA evaluates candidate plans with the shared discrete-event simulation. Search fitness minimizes mean completed waiting time on the first five matched replications, subject to the candidate mean final backlog not exceeding the scenario reference backlog on those same search replications.

For each optimizer seed, the best search plan is then evaluated on the complete matched validation-replication set. Final plan selection keeps only plans whose mean final backlog is no worse than the scenario reference over that complete validation set.

```
In [42]: # Build evenly spaced candidate buses within every study hour.
def build_candidate_stop6_schedule(
    ctx,
    period,
    trip_counts
):
    """Create an evenly spaced candidate Stop 6 schedule from a five-hour service-allocation plan."""
    # Select the five study hours for the requested Morning or Evening optimization period.
    hours = (
        morning_hours
        if period == "Morning"
        else evening_hours
    )

    # Build synthetic Stop 6 bus arrivals from the candidate hourly trip counts.
    records = []

    # Place buses at the midpoints of equal within-hour headway intervals.
    for hour, count in zip(
        hours,
        trip_counts
    ):
        count = int(count)

        if count < 1:
            raise ValueError(
                "Each study hour must contain at least one bus."
            )

        headway = 60.0 / count

        within_hour_times = (
            np.arange(count) + 0.5
        ) * headway

        for trip_number, minute_offset in enumerate(
            within_hour_times,
```

```

        start=1
    ):
        records.append({
            "trip_id":
                f"CAND_{ctx['name']}_{period}_{hour:02d}_{trip_number:02d}",
            "Hour": int(hour),
            "Period": period,
            "Stop 6 Arrival":
                (hour * 60) + float(minute_offset),
            "Hourly Trips": count,
            "Headway Minutes": headway
        })

    return pd.DataFrame(
        records
    )

```

*# Expand each Stop 6 candidate departure to the five modeled stops
using the corresponding hour-specific median GTFS travel offsets.*

```

def expand_candidate_stop_times(
    ctx,
    candidate_schedule
):
    """Expand candidate Stop 6 arrivals to all modeled stops using hour-specific GTFS median offsets."""
    # Expand each synthetic Stop 6 arrival into one arrival event at every modeled stop.
    records = []

    # Apply the hour-specific median GTFS downstream offset for each stop.
    for _, trip in candidate_schedule.iterrows():
        hour = int(
            trip["Hour"]
        )

        for stop in selected_stops:
            records.append({
                "trip_id":
                    trip["trip_id"],
                "Hour":
                    hour,
                "Period":

```

```

        trip["Period"],
        "stop_sequence":
            int(stop),
        "arrival_minutes":
            float(
                trip["Stop 6 Arrival"]
            )
        + float(
            ctx[
                "travel_offset_lookup"
            ][(hour, stop)]
        )
    })

return pd.DataFrame(
    records
)

```

*# Recompute starting occupancy for every candidate hour from the
representative upstream net flow and candidate number of buses.*

```

def create_candidate_initial_occupancy(
    ctx,
    candidate_schedule
):
    """Recalculate candidate-bus starting occupancy from hourly upstream net flow and candidate bus counts."""
    # Determine how many candidate buses share each hour's representative upstream flow.
    candidate_hourly_counts = (
        candidate_schedule
        .groupby("Hour")
        .size()
        .to_dict()
    )

    # Allocate the unchanged hourly upstream net-flow target across the candidate buses.
    records = []

    # Convert the per-bus allocation to a non-negative whole occupancy within capacity.
    for _, trip in candidate_schedule.iterrows():
        hour = int(
            trip["Hour"]

```

```

    )

    bus_count = int(
        candidate_hourly_counts[
            hour
        ]
    )

    unrounded = (
        ctx["upstream_flow_lookup"][hour]
        / bus_count
    )

    initial_occupancy = int(
        np.minimum(
            bus_capacity,
            np.maximum(
                0,
                np.floor(
                    unrounded + 0.5
                )
            )
        )
    )

    records.append({
        "trip_id":
            trip["trip_id"],
        "Hour":
            hour,
        "Period":
            trip["Period"],
        "Initial Occupancy":
            initial_occupancy
    })

return pd.DataFrame(
    records
)

```

```

# Redistribute unchanged stop-hour alighting targets across each
# candidate set while preserving every complete target.
def create_candidate_alighting_plan(
    ctx,
    candidate_stop_times
):
    """Redistribute unchanged stop-hour alighting targets across the candidate buses while preserving each target."""
    # Attach the unchanged stop-hour alighting targets to the candidate service events.
    plan = candidate_stop_times.merge(
        ctx["alighting_targets"][
            [
                "Stop Sequence",
                "Hour",
                "Alighting Target"
            ]
        ],
        left_on=[
            "stop_sequence",
            "Hour"
        ],
        right_on=[
            "Stop Sequence",
            "Hour"
        ],
        how="left"
    )

    if plan[
        "Alighting Target"
    ].isna().any():
        raise ValueError(
            "A candidate stop-hour has no alighting target."
        )

    plan = plan.sort_values(
        [
            "stop_sequence",
            "Hour",
            "arrival_minutes"
        ]
    ).reset_index(drop=True)

```

```

plan[
    "Assigned Alightings"
] = 0

for (_, _), group in plan.groupby(
    ["stop_sequence", "Hour"]
):
    target = int(
        group[
            "Alighting Target"
        ].iloc[0]
    )

    bus_count = len(
        group
    )

    quotient = (
        target // bus_count
    )

    remainder = (
        target % bus_count
    )

    assignments = (
        [quotient + 1] * remainder
        + [quotient]
        * (bus_count - remainder)
    )

    plan.loc[
        group.index,
        "Assigned Alightings"
    ] = assignments

return plan

```

Build the chronological candidate event queue for one period.

```

def create_candidate_event_queue(
    passengers,
    candidate_stop_times,
    period
):
    """Build the chronological event queue for one candidate service plan and study period."""
    # Collect passenger and candidate-bus events in one chronological priority queue.
    events = []

    # Restrict passengers to the requested study period before creating arrival events.
    period_passengers = (
        passengers[
            passengers["Period"]
            == period
        ]
    )

    # Passenger events use priority 1 so an exact-time bus arrival is processed first.
    for _, passenger in period_passengers.iterrows():
        heapq.heappush(
            events,
            (
                float(
                    passenger[
                        "Arrival Minutes"
                    ]
                ),
                1,
                "passenger_arrival",
                int(
                    passenger[
                        "Stop Sequence"
                    ]
                ),
                passenger[
                    "Passenger ID"
                ]
            )
        )

    # Candidate bus events use priority 0 and carry trip and stop information.

```

```

for _, bus in candidate_stop_times.iterrows():
    heapq.heappush(
        events,
        (
            float(
                bus[
                    "arrival_minutes"
                ]
            ),
            0,
            "bus_arrival",
            int(
                bus[
                    "stop_sequence"
                ]
            ),
            bus[
                "trip_id"
            ]
        )
    )

return events

```

```

In [43]: # Validate a five-hour service plan against the scenario's fixed
         # hourly bounds and period budget.
def is_valid_service_plan(
    ctx,
    period,
    trip_counts
):
    """Check a five-hour service plan against the scenario hourly bounds and fixed period budget."""
    # Normalize the candidate allocation to integer hourly trip counts.
    trip_counts = [
        int(value)
        for value in trip_counts
    ]

    # Require one decision value for every study hour.
    if len(trip_counts) != 5:

```

```

        return False

    # Enforce the scenario's hourly minimum and maximum service bounds.
    if any(
        count
        < ctx["min_hourly_trips"]
        or count
        > ctx["max_hourly_trips"]
        for count in trip_counts
    ):
        return False

    if sum(trip_counts) != int(
        ctx["service_budget"][
            period
        ]
    ):
        return False

    return True

# Evaluate one candidate plan on one replication.
def evaluate_candidate_replication(
    ctx,
    period,
    trip_counts,
    passengers,
    replication,
    seed
):
    """Evaluate one candidate service plan on one matched passenger-demand replication."""
    # Normalize the service plan before constructing any candidate service inputs.
    trip_counts = [
        int(value)
        for value in trip_counts
    ]

    if not is_valid_service_plan(
        ctx,
        period,

```

```

        trip_counts
    ):
        raise ValueError(
            "Candidate service plan violates "
            "the scenario constraints."
        )

    # Build the synthetic Stop 6 schedule for the candidate hourly allocation.
    candidate_schedule = (
        build_candidate_stop6_schedule(
            ctx,
            period,
            trip_counts
        )
    )

    # Propagate each candidate bus through the five modeled stops using GTFS-derived offsets.
    candidate_stop_times = (
        expand_candidate_stop_times(
            ctx,
            candidate_schedule
        )
    )

    # Recalculate starting occupancy because the hourly upstream flow is shared by a new bus count.
    candidate_occupancy = (
        create_candidate_initial_occupancy(
            ctx,
            candidate_schedule
        )
    )

    # Redistribute the unchanged alighting targets across the candidate buses.
    candidate_alighting = (
        create_candidate_alighting_plan(
            ctx,
            candidate_stop_times
        )
    )

    occupancy_lookup = (

```

```

        candidate_occupancy
        .set_index(
            "trip_id"
        )["Initial Occupancy"]
        .to_dict()
    )

    candidate_alighting_lookup = (
        candidate_alighting
        .set_index(
            [
                "trip_id",
                "stop_sequence"
            ]
        )["Assigned Alightings"]
        .to_dict()
    )

    # Combine the matched passenger stream with the candidate bus schedule in chronological order.
    events = (
        create_candidate_event_queue(
            passengers,
            candidate_stop_times,
            period
        )
    )

    generated_passengers = int(
        (
            passengers["Period"]
            == period
        ).sum()
    )

    (
        result,
        _,
        _,
        _
    ) = run_service_simulation(

```

```

        period=period,
        events=events,
        generated_passengers=
            generated_passengers,
        replication=int(
            replication
        ),
        seed=int(seed),
        occupancy_lookup=
            occupancy_lookup,
        service_alighting_lookup=
            candidate_alighting_lookup
    )

    result[
        "Service Plan"
    ] = tuple(
        trip_counts
    )

    return result

def evaluate_candidate_replications(
    ctx,
    period,
    trip_counts,
    replication_ids
):
    """Evaluate one candidate service plan across a specified set of matched replications."""
    # Collect one result row for every matched replication used in this evaluation stage.
    records = []

    # Reuse the recorded seed and passenger stream for each matched replication.
    for replication in replication_ids:
        replication = int(
            replication
        )

        seed = int(
            ctx["seed_lookup"][

```

```

        replication
    ]
)

passengers = (
    ctx[
        "passengers_by_replication"
    ][replication]
)

records.append(
    evaluate_candidate_replication(
        ctx,
        period,
        trip_counts,
        passengers,
        replication,
        seed
    )
)

return pd.DataFrame(
    records
)

```

```

In [44]: # Evaluate one candidate plan on the fixed five-replication search set.
# Plans whose mean final backlog exceeds the scenario reference are
# assigned infinite fitness and are therefore infeasible.
def evaluate_service_plan_fitness(
    ctx,
    period,
    trip_counts
):
    """Return the mean completed wait for a feasible candidate plan and reject plans that violate the backlog safeguard."""
    # Use an immutable service-plan key so repeated candidate evaluations can be cached.
    plan = tuple(
        int(value)
        for value in trip_counts
    )

```

```
key = (
    period,
    plan
)

cache = (
    ctx[
        "fitness_cache"
    ]
)

if key not in cache:
    results = (
        evaluate_candidate_replications(
            ctx,
            period,
            plan,
            ctx[
                "search_replication_ids"
            ]
        )
    )

    mean_wait = float(
        results[
            "Mean Completed Wait"
        ].mean()
    )

    mean_backlog = float(
        results[
            "Final Backlog"
        ].mean()
    )

    mean_peak_queue = float(
        results[
            "Peak Queue"
        ].mean()
    )
```

```

    mean_p95_wait = float(
        results[
            "P95 Completed Wait"
        ].mean()
    )

    feasible = bool(
        mean_backlog
        <= ctx[
            "feasibility_backlog"
        ][period]
        + 1e-9
    )

    cache[key] = {
        "Period":
            period,
        "Service Plan":
            plan,
        "Fitness":
            mean_wait
            if feasible
            else np.inf,
        "Mean Wait":
            mean_wait,
        "Mean Backlog":
            mean_backlog,
        "Mean Peak Queue":
            mean_peak_queue,
        "Mean P95 Wait":
            mean_p95_wait,
        "Feasible":
            feasible
    }

    return cache[
        key
    ]

```

```

def random_valid_plan(

```

```

    ctx,
    period,
    rng
):
    """Generate a random five-hour service plan that satisfies the fixed budget and hourly bounds."""
    # Start from the scenario-specific fixed Morning or Evening trip budget.
    budget = int(
        ctx[
            "service_budget"
        ][period]
    )

    lower = int(
        ctx[
            "min_hourly_trips"
        ]
    )

    upper = int(
        ctx[
            "max_hourly_trips"
        ]
    )

    while True:
        plan = rng.integers(
            lower,
            upper + 1,
            size=5
        )

        if int(
            plan.sum()
        ) == budget:
            return tuple(
                int(value)
                for value in plan
            )

def repair_service_plan(

```

```

ctx,
period,
values,
rng
):
    """Repair a proposed service allocation so it satisfies the hourly bounds and fixed period budget."""
    lower = int(
        ctx[
            "min_hourly_trips"
        ]
    )

    upper = int(
        ctx[
            "max_hourly_trips"
        ]
    )

    budget = int(
        ctx[
            "service_budget"
        ][period]
    )

    # Convert the proposed values to integer hourly trip counts before repair.
    plan = np rint(
        np.array(
            values,
            dtype=float
        )
    ).astype(int)

    # Enforce the hourly bounds before correcting the total service budget.
    plan = np.clip(
        plan,
        lower,
        upper
    )

    while plan.sum() < budget:
        choices = np.where(

```

```

        plan < upper
    )[0]

    if len(choices) == 0:
        raise RuntimeError(
            "Unable to repair service plan upward."
        )

    plan[
        int(
            rng.choice(
                choices
            )
        )
    ] += 1

while plan.sum() > budget:
    choices = np.where(
        plan > lower
    )[0]

    if len(choices) == 0:
        raise RuntimeError(
            "Unable to repair service plan downward."
        )

    plan[
        int(
            rng.choice(
                choices
            )
        )
    ] -= 1

return tuple(
    int(value)
    for value in plan
)

def neighboring_service_plan(

```

```

ctx,
period,
plan,
rng
):
    """Create a nearby feasible service plan by moving one trip between study hours."""
    lower = int(
        ctx[
            "min_hourly_trips"
        ]
    )

    upper = int(
        ctx[
            "max_hourly_trips"
        ]
    )

    # Work on a copy so the original candidate plan is not modified in place.
    plan = np.array(
        plan,
        dtype=int
    )

    donors = np.where(
        plan > lower
    )[0]

    receivers = np.where(
        plan < upper
    )[0]

    if (
        len(donors) == 0
        or len(receivers) == 0
    ):
        return tuple(
            int(value)
            for value in plan
        )

```

```

for _ in range(30):
    donor = int(
        rng.choice(
            donors
        )
    )

    receiver = int(
        rng.choice(
            receivers
        )
    )

    if donor != receiver:
        plan[donor] -= 1
        plan[receiver] += 1

    return tuple(
        int(value)
        for value in plan
    )

return tuple(
    int(value)
    for value in plan
)

```

In [45]: *# Run the Genetic Algorithm for one study period and optimizer seed.*

```

def run_genetic_algorithm(
    ctx,
    period,
    optimizer_seed,
    evaluation_budget=evaluation_budget,
    population_size=ga_population_size
):
    """Run the Genetic Algorithm for one period and optimizer seed under the fixed unique-plan evaluation budget."""
    # Use a dedicated optimizer seed so the GA search is reproducible.
    rng = np.random.default_rng(
        optimizer_seed
    )

```

```

# Seed the initial population with feasible service plans only.
population = [
    tuple(
        int(value)
        for value in ctx[
            "reference_plan_lookup"
        ][period]
    )
]

while len(population) < population_size:
    plan = random_valid_plan(
        ctx,
        period,
        rng
    )

    if plan not in population:
        population.append(
            plan
        )

seen_plans = set()
best_result = None
history = []

# Continue until the agreed number of unique candidate plans has been evaluated.
while len(seen_plans) < evaluation_budget:
    # Score each unseen feasible candidate using the fixed search-replication set.
    scored = []

    for plan in population:
        if (
            plan in seen_plans
            and len(seen_plans)
            >= evaluation_budget
        ):
            continue

    result = (

```

```

        evaluate_service_plan_fitness(
            ctx,
            period,
            plan
        )
    )

    seen_plans.add(
        plan
    )

    scored.append(
        (
            result["Fitness"],
            plan,
            result
        )
    )

    if (
        best_result is None
        or result["Fitness"]
        < best_result["Fitness"]
    ):
        best_result = result

    if len(
        seen_plans
    ) >= evaluation_budget:
        break

    if best_result is None:
        raise RuntimeError(
            "GA did not evaluate a service plan."
        )

    history.append({
        "Technique":
            "GA",
        "Scenario":
            ctx["name"],
    })

```

```

        "Period":
            period,
        "Optimizer Seed":
            int(optimizer_seed),
        "Unique Plans Evaluated":
            len(seen_plans),
        "Best Fitness":
            best_result["Fitness"]
    })

    if len(
        seen_plans
    ) >= evaluation_budget:
        break

    scored = sorted(
        scored,
        key=lambda item:
            item[0]
    )

    if len(scored) == 0:
        raise RuntimeError(
            "GA population produced no scorable plans."
        )

    # Preserve the best current plans before generating the next population.
    elites = [
        scored[0][1],
        scored[
            min(
                1,
                len(scored) - 1
            )
        ][1]
    ]

    # Select parents by tournament so better plans are favored while retaining search diversity.
    def tournament_select():
        """Select one GA parent by comparing a random subset of the current scored population."""
        indexes = rng.choice(

```

```

        len(scored),
        size=min(
            3,
            len(scored)
        ),
        replace=False
    )

    competitors = [
        scored[
            int(index)
        ]
        for index in indexes
    ]

    return min(
        competitors,
        key=lambda item:
            item[0]
    )[1]

next_population = (
    elites.copy()
)

while (
    len(next_population)
    < population_size
):
    parent1 = np.array(
        tournament_select()
    )

    parent2 = np.array(
        tournament_select()
    )

    mask = (
        rng.random(5)
        < 0.5
    )

```

```

        child = np.where(
            mask,
            parent1,
            parent2
        )

        child = (
            repair_service_plan(
                ctx,
                period,
                child,
                rng
            )
        )

        if rng.random() < 0.5:
            child = (
                neighboring_service_plan(
                    ctx,
                    period,
                    child,
                    rng
                )
            )

        next_population.append(
            child
        )

    population = (
        next_population
    )

    return (
        best_result,
        pd.DataFrame(
            history
        )
    )
)

```

```

# Run all optimizer seeds for one scenario, validate each search winner
# on the complete matched replication set, and select the final plan.
def run_ga_for_scenario(
    ctx,
    optimizer_seeds,
    evaluation_budget,
    final_backlog_reference=None,
    population_size=ga_population_size
):
    """Run all GA optimizer seeds for one scenario, validate each winner, and select the final feasible plan by mean wait."""
    # Keep the full-validation result of every optimizer seed for both study periods.
    run_records = []

    # Search Morning and Evening independently because their fixed service budgets differ.
    for period in [
        "Morning",
        "Evening"
    ]:
        for optimizer_seed in optimizer_seeds:
            search_best, _ = (
                run_genetic_algorithm(
                    ctx,
                    period,
                    optimizer_seed,
                    evaluation_budget,
                    population_size
                )
            )

        # Re-evaluate each search winner on the complete matched validation-replication set.
        validated = (
            evaluate_candidate_replications(
                ctx,
                period,
                search_best[
                    "Service Plan"
                ],
                ctx[
                    "validation_replication_ids"
                ]
            )

```

```

    )
)

run_records.append({
    "Scenario":
        ctx["name"],
    "Period":
        period,
    "Technique":
        "GA",
    "Optimizer Seed":
        int(optimizer_seed),
    "Evaluation Budget":
        int(evaluation_budget),
    "Search Best Fitness":
        float(
            search_best[
                "Fitness"
            ]
        ),
    "Best Service Plan":
        search_best[
            "Service Plan"
        ],
    "Validated Mean Wait":
        float(
            validated[
                "Mean Completed Wait"
            ].mean()
        ),
    "Validated Mean Backlog":
        float(
            validated[
                "Final Backlog"
            ].mean()
        ),
    "Validated Mean Peak Queue":
        float(
            validated[
                "Peak Queue"
            ].mean()
        )
})

```

```

    ),
    "Validated Mean P95 Wait":
        float(
            validated[
                "P95 Completed Wait"
            ].mean()
        ),
    "Validated Mean Period-End Queue":
        float(
            validated[
                "Period-End Queue"
            ].mean()
        ),
    "Validated Mean Bus Occupancy":
        float(
            validated[
                "Mean Bus Occupancy"
            ].mean()
        ),
    "Validated Max Bus Occupancy":
        int(
            validated[
                "Maximum Bus Occupancy"
            ].max()
        ),
    "Passenger Conservation":
        bool(
            validated[
                "Passenger Conservation"
            ].all()
        )
    })

```

Consolidate all optimizer-seed winners before final plan selection.

```

runs_df = pd.DataFrame(
    run_records
)

```

```

selected_records = []

```

```

for period in [

```

```

    "Morning",
    "Evening"
]:
    period_runs = runs_df.loc[
        runs_df["Period"]
        == period
    ].copy()

    # Baseline reproduction has no new stress-scenario safeguard; stress scenarios use their own reference backlog.
    if final_backlog_reference is None:
        feasible = (
            period_runs

            safeguard_available = False

        else:
            feasible = period_runs.loc[
                period_runs[
                    "Validated Mean Backlog"
                ]
                <= float(
                    final_backlog_reference[
                        period
                    ]
                )
                + 1e-9
            ].copy()

            safeguard_available = True

    if len(feasible) == 0:
        raise RuntimeError(
            f"No {ctx['name']} {period} GA plan "
            "passed the full-replication final-backlog safeguard."
        )

    # Among safeguard-feasible plans, select the lowest validated mean completed wait.
    best_row = feasible.loc[
        feasible[
            "Validated Mean Wait"

```

```

    ].idxmin()
]

selected_records.append({
    "Scenario":
        ctx["name"],
    "Period":
        period,
    "Technique":
        "GA",
    "Service Plan":
        best_row[
            "Best Service Plan"
        ],
    "Mean Wait":
        float(
            best_row[
                "Validated Mean Wait"
            ]
        ),
    "Mean Backlog":
        float(
            best_row[
                "Validated Mean Backlog"
            ]
        ),
    "Mean Peak Queue":
        float(
            best_row[
                "Validated Mean Peak Queue"
            ]
        ),
    "Mean P95 Wait":
        float(
            best_row[
                "Validated Mean P95 Wait"
            ]
        ),
    "Mean Period-End Queue":
        float(
            best_row[

```

```

        "Validated Mean Period-End Queue"
    ],
),
"Mean Bus Occupancy":
    float(
        best_row[
            "Validated Mean Bus Occupancy"
        ]
    ),
"Maximum Bus Occupancy":
    int(
        best_row[
            "Validated Max Bus Occupancy"
        ]
    ),
"Passed Final Backlog Safeguard":
    (
        True
        if safeguard_available
        else np.nan
    )
})

return (
    runs_df,
    pd.DataFrame(
        selected_records
    )
)

```

21. Baseline GA Reproduction Check

Before interpreting stress-scenario optimization, the generalized GA is rerun on Baseline. The result is compared against the documented Midterm GA plans and validated mean waiting times. Matching values confirm that the current generalized candidate evaluator and GA reproduce the established Baseline optimization result.

```

In [46]: # Run the Baseline GA using the original search design.
ga_core_runs, ga_core_selected = (

```

```

run_ga_for_scenario(
    baseline_ctx,
    optimizer_seeds=
        optimizer_seeds,
    evaluation_budget=
        evaluation_budget,
    final_backlog_reference=None
)
)

# Regression targets documented by the completed Midterm GA experiment.
midterm_ga_reference = pd.DataFrame([
    {
        "Period": "Morning",
        "Midterm Plan":
            (8, 9, 11, 9, 6),
        "Midterm Mean Wait":
            3.488068
    },
    {
        "Period": "Evening",
        "Midterm Plan":
            (11, 7, 9, 10, 7),
        "Midterm Mean Wait":
            3.532199
    }
])

check_rows = []

for _, expected in (
    midterm_ga_reference
    .iterrows()
):
    period = expected[
        "Period"
    ]

    actual = ga_core_selected.loc[
        ga_core_selected[
            "Period"

```

```

    ] == period
].iloc[0]

actual_plan = tuple(
    int(value)
    for value in actual[
        "Service Plan"
    ]
)

expected_plan = tuple(
    int(value)
    for value in expected[
        "Midterm Plan"
    ]
)

actual_wait = float(
    actual[
        "Mean Wait"
    ]
)

expected_wait = float(
    expected[
        "Midterm Mean Wait"
    ]
)

check_rows.append({
    "Period":
        period,
    "Generalized Plan":
        actual_plan,
    "Midterm Plan":
        expected_plan,
    "Plan Matches":
        actual_plan
        == expected_plan,
    "Generalized Mean Wait":
        actual_wait,

```

```

        "Midterm Mean Wait":
            expected_wait,
        "Wait Matches":
            bool(
                np.isclose(
                    actual_wait,
                    expected_wait,
                    atol=1e-6
                )
            )
    })

baseline_reproduction_check = (
    pd.DataFrame(
        check_rows
    )
)

baseline_reproduction_ok = bool(
    baseline_reproduction_check[
        "Plan Matches"
    ].all()
    and baseline_reproduction_check[
        "Wait Matches"
    ].all()
)

assert baseline_reproduction_ok, (
    "Generalized GA did not reproduce the documented "
    "Midterm Baseline optimization result."
)

print(
    "Generalized GA reproduces the Midterm Baseline result:",
    baseline_reproduction_ok
)

baseline_reproduction_check

```

Generalized GA reproduces the Midterm Baseline result: True

Out[46]:

	Period	Generalized Plan	Midterm Plan	Plan Matches	Generalized Mean Wait	Midterm Mean Wait	Wait Matches
0	Morning	(8, 9, 11, 9, 6)	(8, 9, 11, 9, 6)	True	3.488068	3.488068	True
1	Evening	(11, 7, 9, 10, 7)	(11, 7, 9, 10, 7)	True	3.532199	3.532199	True

22. Optimize High-Demand, Reduced-Service, and Combined

High-Demand keeps the Baseline service budget and 5-to-12 bounds because only selected-stop boarding demand changes.

Reduced-Service keeps the reduced 38/39 trip budgets and searches within 4-to-12 trips per hour. The 4 lower bound includes the minimum hourly service in the reduced reference, while the 12 upper bound preserves the highest hourly service level observed in the verified original GTFS.

Combined uses the same High-Demand passenger streams as High-Demand together with the Reduced-Service 38/39 trip budgets and 4-to-12 bounds. Upstream flow and alighting targets are not multiplied by 1.40 because the demand increase is limited to boarding demand at the five modeled stops.

```
In [47]: # Build full-validation final-backlog safeguard thresholds from each
# scenario's own reference-service results.
high_demand_reference_backlog = (
    build_backlog_reference(
        high_demand_results_df,
        high_demand_validation_ids
    )
)

reduced_service_reference_backlog = (
    build_backlog_reference(
        reduced_service_results_df,
        reduced_service_validation_ids
    )
)

combined_reference_backlog = (
    build_backlog_reference(
```

```

        combined_results_df,
        combined_validation_ids
    )
)

# Run the same GA structure on each stress scenario.
high_demand_runs, high_demand_selected = (
    run_ga_for_scenario(
        high_demand_ctx,
        optimizer_seeds=
            optimizer_seeds,
        evaluation_budget=
            evaluation_budget,
        final_backlog_reference=
            high_demand_reference_backlog
    )
)

reduced_service_runs, reduced_service_selected = (
    run_ga_for_scenario(
        reduced_service_ctx,
        optimizer_seeds=
            optimizer_seeds,
        evaluation_budget=
            evaluation_budget,
        final_backlog_reference=
            reduced_service_reference_backlog
    )
)

combined_runs, combined_selected = (
    run_ga_for_scenario(
        combined_ctx,
        optimizer_seeds=
            optimizer_seeds,
        evaluation_budget=
            evaluation_budget,
        final_backlog_reference=
            combined_reference_backlog
    )
)

```

```

selected_ga_plans = pd.concat(
    [
        ga_core_selected,
        high_demand_selected,
        reduced_service_selected,
        combined_selected
    ],
    ignore_index=True
)

```

selected_ga_plans

Out[47]:

	Scenario	Period	Technique	Service Plan	Mean Wait	Mean Backlog	Mean Peak Queue	Mean P95 Wait	Mean Period-End Queue	Mean Bus Occupancy	Maximum Bus Occupancy	Passed Final Backlog Safeguard
0	Baseline	Morning	GA	(8, 9, 11, 9, 6)	3.488068	1.833333	17.500000	7.146191	2.933333	15.673643	36	NaN
1	Baseline	Evening	GA	(11, 7, 9, 10, 7)	3.532199	0.966667	8.633333	7.473238	2.166667	12.119091	28	NaN
2	High-Demand	Morning	GA	(8, 12, 10, 8, 5)	3.476655	2.666667	21.866667	7.519975	6.066667	17.882171	42	1.0
3	High-Demand	Evening	GA	(10, 10, 10, 9, 5)	3.500153	1.566667	11.200000	7.781034	4.166667	14.010000	27	1.0
4	Reduced-Service	Morning	GA	(6, 9, 11, 7, 5)	3.897731	2.100000	18.566667	9.008420	4.466667	17.623684	37	1.0
5	Reduced-Service	Evening	GA	(8, 8, 9, 10, 4)	3.967706	1.600000	8.833333	8.766209	3.833333	13.760513	27	1.0
6	Combined	Morning	GA	(7, 11, 8, 6, 6)	4.010456	2.266667	24.366667	8.615005	3.900000	20.035088	49	1.0
7	Combined	Evening	GA	(7, 8, 9, 8, 7)	4.002511	1.233333	12.366667	7.798894	2.833333	15.635043	37	1.0

23. Selected-Plan Validation and Stop-Level Results

Each selected stress-scenario GA plan is re-evaluated on the full matched validation-replication set. The checks confirm the fixed service budget, hourly bounds, final-backlog safeguard, passenger conservation, non-negative completed waiting times, and bus occupancy within the modeled 0-to-60 range.

Stop-level outputs are also retained for Stops 6 through 10 so the effect of each selected service allocation can be inspected beyond the period-level averages.

```
In [48]: # Verify the selected GA plans against the explicit scenario safeguards.
def check_scenario_safeguards(
    ctx,
    selected_df,
    runs_df,
    reference_backlog
):
    """Verify selected GA plans against budget, bound, backlog, and passenger-conservation safeguards."""
    # Build an explicit reviewer-facing validation row for each selected period plan.
    rows = []

    # Check service-budget, hourly-bound, backlog, and conservation requirements together.
    for _, row in selected_df.iterrows():
        period = row[
            "Period"
        ]

        plan = tuple(
            int(value)
            for value in row[
                "Service Plan"
            ]
        )

        rows.append({
            "Scenario":
                ctx["name"],
            "Period":
```

```

        period,
    "Plan":
        plan,
    "Plan Backlog":
        float(
            row[
                "Mean Backlog"
            ]
        ),
    "Reference Backlog":
        float(
            reference_backlog[
                period
            ]
        ),
    "Budget OK":
        sum(plan)
        == int(
            ctx[
                "service_budget"
            ][period]
        ),
    "Bounds OK":
        all(
            ctx[
                "min_hourly_trips"
            ]
            <= count
            <= ctx[
                "max_hourly_trips"
            ]
            for count in plan
        ),
    "Backlog Safeguard OK":
        float(
            row[
                "Mean Backlog"
            ]
        )
        <= float(
            reference_backlog[

```

```

        period
    ]
)
+ 1e-9,
"Passenger Conservation OK":
    bool(
        runs_df.loc[
            runs_df[
                "Period"
            ] == period,
            "Passenger Conservation"
        ].all()
    )
})

return pd.DataFrame(
    rows
)

scenario_safeguards = pd.concat(
    [
        check_scenario_safeguards(
            high_demand_ctx,
            high_demand_selected,
            high_demand_runs,
            high_demand_reference_backlog
        ),
        check_scenario_safeguards(
            reduced_service_ctx,
            reduced_service_selected,
            reduced_service_runs,
            reduced_service_reference_backlog
        ),
        check_scenario_safeguards(
            combined_ctx,
            combined_selected,
            combined_runs,
            combined_reference_backlog
        )
    ],

```

```

        ignore_index=True
    )

    safeguard_columns = [
        "Budget OK",
        "Bounds OK",
        "Backlog Safeguard OK",
        "Passenger Conservation OK"
    ]

    all_safeguards_ok = bool(
        scenario_safeguards[
            safeguard_columns
        ].values.all()
    )

    assert all_safeguards_ok, (
        "At least one selected GA plan failed a required safeguard."
    )

    print(
        "All stress-scenario GA safeguards passed:",
        all_safeguards_ok
    )

    scenario_safeguards

```

All stress-scenario GA safeguards passed: True

Out[48]:

	Scenario	Period	Plan	Plan Backlog	Reference Backlog	Budget OK	Bounds OK	Backlog Safeguard OK	Passenger Conservation OK
0	High-Demand	Morning	(8, 12, 10, 8, 5)	2.666667	2.866667	True	True	True	True
1	High-Demand	Evening	(10, 10, 10, 9, 5)	1.566667	4.266667	True	True	True	True
2	Reduced-Service	Morning	(6, 9, 11, 7, 5)	2.100000	2.166667	True	True	True	True
3	Reduced-Service	Evening	(8, 8, 9, 10, 4)	1.600000	3.233333	True	True	True	True
4	Combined	Morning	(7, 11, 8, 6, 6)	2.266667	2.866667	True	True	True	True
5	Combined	Evening	(7, 8, 9, 8, 7)	1.233333	4.266667	True	True	True	True

In [49]:

```
# Evaluate one selected plan in detail and retain period-level plus
# stop-level outputs for every validation replication.
def evaluate_selected_plan_detailed(
    ctx,
    period,
    trip_counts,
    replication_ids
):
    """Evaluate one selected GA plan across validation replications and retain period-level and stop-level results."""
    # Keep the official study-period boundary separate from later eligible bus events.
    end_time = (
        11 * 60
        if period == "Morning"
        else 20 * 60
    )

    plan = [
        int(value)
        for value in trip_counts
    ]
```

```

# Retain both period-level and stop-level outputs for every validation replication.
per_replication_rows = []
stop_rows = []

# Rebuild the same candidate service inputs for each matched passenger realization.
for replication in replication_ids:
    replication = int(
        replication
    )

    seed = int(
        ctx[
            "seed_lookup"
        ][replication]
    )

    passengers = (
        ctx[
            "passengers_by_replication"
        ][replication]
    )

    candidate_schedule = (
        build_candidate_stop6_schedule(
            ctx,
            period,
            plan
        )
    )

    candidate_stop_times = (
        expand_candidate_stop_times(
            ctx,
            candidate_schedule
        )
    )

    candidate_occupancy = (
        create_candidate_initial_occupancy(
            ctx,

```

```

        candidate_schedule
    )
)

candidate_alighting = (
    create_candidate_alighting_plan(
        ctx,
        candidate_stop_times
    )
)

occupancy_lookup = (
    candidate_occupancy
    .set_index(
        "trip_id"
    )["Initial Occupancy"]
    .to_dict()
)

alighting_lookup = (
    candidate_alighting
    .set_index(
        [
            "trip_id",
            "stop_sequence"
        ]
    )["Assigned Alightings"]
    .to_dict()
)

events = (
    create_candidate_event_queue(
        passengers,
        candidate_stop_times,
        period
    )
)

generated = int(
    (
        passengers["Period"]

```

```

        == period
    ).sum()
)

(
    result,
    waiting_df,
    bus_history_df,
    queue_history_df,
    -
) = run_service_simulation(
    period=period,
    events=events,
    generated_passengers=
        generated,
    replication=
        replication,
    seed=
        seed,
    occupancy_lookup=
        occupancy_lookup,
    service_alighting_lookup=
        alighting_lookup
)

# Capture lower-bound checks used to validate waiting-time and occupancy state values.
minimum_wait = float(
    waiting_df[
        "Waiting Time"
    ].min()
)

minimum_occupancy = int(
    bus_history_df[
        "Occupancy After"
    ].min()
)

maximum_occupancy = int(
    bus_history_df[
        "Occupancy After"

```

```

    ].max()
)

row = dict(
    result
)

row[
    "Scenario"
] = ctx["name"]

row[
    "Service Plan"
] = tuple(
    plan
)

row[
    "Minimum Completed Wait"
] = minimum_wait

row[
    "Minimum Bus Occupancy"
] = minimum_occupancy

row[
    "Waiting Time Valid"
] = bool(
    minimum_wait >= 0
)

row[
    "Bus Occupancy Valid"
] = bool(
    minimum_occupancy >= 0
    and maximum_occupancy
    <= bus_capacity
)

per_replication_rows.append(
    row

```

```

)

# Summarize waiting and queue behavior separately for each modeled stop.
for stop in selected_stops:
    waits = waiting_df.loc[
        waiting_df[
            "Stop Sequence"
        ] == stop,
        "Waiting Time"
    ]

    queue_stop = (
        queue_history_df.loc[
            queue_history_df[
                "Stop Sequence"
            ] == stop
        ]
    )

    queue_end = (
        queue_stop.loc[
            queue_stop[
                "Time"
            ] <= end_time
        ]
    )

    stop_rows.append({
        "Scenario":
            ctx["name"],
        "Period":
            period,
        "Replication":
            replication,
        "Seed":
            seed,
        "Stop Sequence":
            int(stop),
        "Mean Wait":
            float(
                waits.mean()
            )
    })

```

```

        )
        if len(waits)
        else 0.0,
    "P95 Wait":
        float(
            waits.quantile(
                0.95
            )
        )
        if len(waits)
        else 0.0,
    "Peak Queue":
        int(
            queue_stop[
                "Queue Length"
            ].max()
        )
        if len(queue_stop)
        else 0,
    "Period-End Queue":
        int(
            queue_end[
                "Queue Length"
            ].iloc[-1]
        )
        if len(queue_end)
        else 0,
    "Final Backlog":
        int(
            queue_stop[
                "Queue Length"
            ].iloc[-1]
        )
        if len(queue_stop)
        else 0
    })

return (
    pd.DataFrame(
        per_replication_rows
    ),

```

```

        pd.DataFrame(
            stop_rows
        )
    )

def evaluate_scenario_selected_plans(
    ctx,
    selected_df
):
    """Evaluate the selected Morning and Evening GA plans for one scenario."""
    # Combine detailed Morning and Evening validation results for this scenario.
    replication_frames = []
    stop_frames = []

    # Evaluate each selected period plan on the scenario's full validation set.
    for _, row in selected_df.iterrows():
        per_replication, stop_level = (
            evaluate_selected_plan_detailed(
                ctx,
                row["Period"],
                row["Service Plan"],
                ctx[
                    "validation_replication_ids"
                ]
            )
        )

        replication_frames.append(
            per_replication
        )

        stop_frames.append(
            stop_level
        )

    return (
        pd.concat(
            replication_frames,
            ignore_index=True
        ),

```

```

        pd.concat(
            stop_frames,
            ignore_index=True
        )
    )

```

In [50]: *# Evaluate the three stress-scenario selected GA plans in full detail.*

```

high_demand_selected_reps, high_demand_stop_level = (
    evaluate_scenario_selected_plans(
        high_demand_ctx,
        high_demand_selected
    )
)

reduced_service_selected_reps, reduced_service_stop_level = (
    evaluate_scenario_selected_plans(
        reduced_service_ctx,
        reduced_service_selected
    )
)

combined_selected_reps, combined_stop_level = (
    evaluate_scenario_selected_plans(
        combined_ctx,
        combined_selected
    )
)

selected_plan_replications = pd.concat(
    [
        high_demand_selected_reps,
        reduced_service_selected_reps,
        combined_selected_reps
    ],
    ignore_index=True
)

selected_plan_stop_level = pd.concat(
    [
        high_demand_stop_level,

```

```

        reduced_service_stop_level,
        combined_stop_level
    ],
    ignore_index=True
)

validation_rows = []

for scenario_name, reps_df in [
    (
        "High-Demand",
        high_demand_selected_reps
    ),
    (
        "Reduced-Service",
        reduced_service_selected_reps
    ),
    (
        "Combined",
        combined_selected_reps
    )
]:
    validation_rows.append({
        "Scenario":
            scenario_name,
        "Replications Used":
            int(
                reps_df[
                    "Replication"
                ].nunique()
            ),
        "Non-Negative Waiting Time Passed":
            bool(
                reps_df[
                    "Waiting Time Valid"
                ].all()
            ),
        "Bus Occupancy In [0, 60] Passed":
            bool(
                reps_df[
                    "Bus Occupancy Valid"

```

```

        ].all()
    ),
    "Passenger Conservation Passed":
        bool(
            reps_df[
                "Passenger Conservation"
            ].all()
        ),
    "Minimum Observed Waiting Time":
        float(
            reps_df[
                "Minimum Completed Wait"
            ].min()
        ),
    "Minimum Observed Bus Occupancy":
        int(
            reps_df[
                "Minimum Bus Occupancy"
            ].min()
        ),
    "Maximum Observed Bus Occupancy":
        int(
            reps_df[
                "Maximum Bus Occupancy"
            ].max()
        )
    })

selected_validation_summary = (
    pd.DataFrame(
        validation_rows
    )
)

validation_pass_columns = [
    "Non-Negative Waiting Time Passed",
    "Bus Occupancy In [0, 60] Passed",
    "Passenger Conservation Passed"
]

all_validation_passed = bool(

```

```

selected_validation_summary[
    validation_pass_columns
].values.all()
)

assert all_validation_passed, (
    "At least one selected GA plan failed the simulation validation checks."
)

print(
    "All selected-plan validation checks passed:",
    all_validation_passed
)

selected_validation_summary

```

All selected-plan validation checks passed: True

Out[50]:

	Scenario	Replications Used	Non-Negative Waiting Time Passed	Bus Occupancy In [0, 60] Passed	Passenger Conservation Passed	Minimum Observed Waiting Time	Minimum Observed Bus Occupancy	Maximum Observed Bus Occupancy
0	High-Demand	30	True	True	True	0.000167	1	42
1	Reduced-Service	30	True	True	True	0.000168	1	37
2	Combined	30	True	True	True	0.000074	1	49

24. Precision and Paired Reference-versus-GA Comparison

The selected GA plans are checked with the same 95% confidence-interval precision rule used by the reference simulation. Reference and GA outputs are then matched on Replication, Seed, and Period.

The paired difference is defined as **GA minus Reference**. Negative values favor the GA plan. A reduction is marked as statistically supported only when the complete two-sided 95% confidence interval is below zero.

Reference buses use the published or scenario-reduced GTFS timings, while GA candidates use midpoint-spaced buses within each hour plus median GTFS downstream travel offsets. Therefore, observed differences reflect both hourly service redistribution and within-hour headway regularization.

```
In [51]: # Apply the project precision rule to each selected stress-scenario GA plan.
precision_frames = []

for scenario_name, reps_df in [
    (
        "High-Demand",
        high_demand_selected_reps
    ),
    (
        "Reduced-Service",
        reduced_service_selected_reps
    ),
    (
        "Combined",
        combined_selected_reps
    )
]:
    precision_df, precision_ok = (
        compute_precision_check(
            reps_df
        )
    )

    precision_df.insert(
        0,
        "Scenario",
        scenario_name
    )

    precision_frames.append(
        precision_df
    )

    print(
        f"{scenario_name}: "
```

```

        f"precision requirement met = {precision_ok}"
    )

selected_precision_check = pd.concat(
    precision_frames,
    ignore_index=True
)

assert selected_precision_check[
    "Precision Passed"
].all(), (
    "At least one selected GA plan does not meet "
    "the required replication precision."
)

selected_precision_check.round(4)

```

High-Demand: precision requirement met = True
 Reduced-Service: precision requirement met = True
 Combined: precision requirement met = True

Out[51]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half- Width	Required Half-Width	Precision Passed
0	High-Demand	Morning	Mean Completed Wait	30	3.4767	0.0822	3.4460	3.5074	0.0307	1.0000	True
1	High-Demand	Morning	Peak Queue	30	21.8667	2.4174	20.9640	22.7693	0.9027	2.1867	True
2	High-Demand	Evening	Mean Completed Wait	30	3.5002	0.1022	3.4620	3.5383	0.0382	1.0000	True
3	High-Demand	Evening	Peak Queue	30	11.2000	2.0240	10.4442	11.9558	0.7558	1.1200	True
4	Reduced-Service	Morning	Mean Completed Wait	30	3.8977	0.1320	3.8485	3.9470	0.0493	1.0000	True
5	Reduced-Service	Morning	Peak Queue	30	18.5667	2.2997	17.7080	19.4254	0.8587	1.8567	True
6	Reduced-Service	Evening	Mean Completed Wait	30	3.9677	0.1701	3.9042	4.0312	0.0635	1.0000	True
7	Reduced-Service	Evening	Peak Queue	30	8.8333	1.2617	8.3622	9.3045	0.4711	1.0000	True
8	Combined	Morning	Mean Completed Wait	30	4.0105	0.0819	3.9799	4.0410	0.0306	1.0000	True
9	Combined	Morning	Peak Queue	30	24.3667	2.3116	23.5035	25.2298	0.8632	2.4367	True
10	Combined	Evening	Mean Completed Wait	30	4.0025	0.1303	3.9538	4.0512	0.0487	1.0000	True
11	Combined	Evening	Peak Queue	30	12.3667	2.1088	11.5792	13.1541	0.7874	1.2367	True

```

In [52]: # Calculate paired GA-minus-reference confidence intervals.
def paired_reference_vs_ga(
    ga_per_replication,
    reference_df,
    scenario_name,
    metrics,
    confidence=0.95
):
    """Compute paired GA-minus-reference confidence intervals for matched replications."""
    # Produce one paired-comparison record per period and evaluation metric.
    rows = []

    # Compare Morning and Evening separately on their matched replication seeds.
    for period in [
        "Morning",
        "Evening"
    ]:
        ga_period = (
            ga_per_replication.loc[
                ga_per_replication[
                    "Period"
                ] == period
            ]
        )

        reference_period = (
            reference_df.loc[
                reference_df[
                    "Period"
                ] == period
            ]
        )

        for metric in metrics:
            # Match GA and reference rows by replication and seed before taking differences.
            merged = (
                ga_period[
                    [
                        "Replication",
                        "Seed",

```

```

        metric
    ]
]
.merge(
    reference_period[
        [
            "Replication",
            "Seed",
            metric
        ]
    ],
    on=[
        "Replication",
        "Seed"
    ],
    suffixes=(
        " GA",
        " Ref"
    )
)
)

n = len(
    merged
)

if n < 2:
    raise ValueError(
        "At least two matched replications "
        "are required for paired comparison."
    )

# Define improvement as GA minus reference, so negative values favor the GA plan.
difference = (
    merged[
        metric + " GA"
    ]
    - merged[
        metric + " Ref"
    ]
)

```

```

mean_difference = (
    difference.mean()
)

standard_deviation = (
    difference.std(
        ddof=1
    )
)

standard_error = (
    standard_deviation
    / np.sqrt(n)
)

# Use the Student t critical value because the paired mean difference is estimated from replications.
t_critical = t.ppf(
    confidence
    + (
        1 - confidence
    ) / 2,
    df=n - 1
)

half_width = (
    t_critical
    * standard_error
)

lower = (
    mean_difference
    - half_width
)

upper = (
    mean_difference
    + half_width
)

rows.append({

```

```

        "Scenario":
            scenario_name,
        "Period":
            period,
        "Metric":
            metric,
        "N":
            n,
        "Mean GA":
            float(
                merged[
                    metric + " GA"
                ].mean()
            ),
        "Mean Reference":
            float(
                merged[
                    metric + " Ref"
                ].mean()
            ),
        "Mean Paired Diff (GA - Reference)":
            float(
                mean_difference
            ),
        "CI Lower":
            float(lower),
        "CI Upper":
            float(upper),
        "Statistically Supported Improvement":
            bool(
                upper < 0
            )
    })

return pd.DataFrame(
    rows
)

```

```

paired_metrics = [
    "Mean Completed Wait",

```

```

        "P95 Completed Wait",
        "Peak Queue",
        "Final Backlog"
    ]

scenario_paired_ci = pd.concat(
    [
        paired_reference_vs_ga(
            high_demand_selected_reps,
            high_demand_results_df,
            "High-Demand",
            paired_metrics
        ),
        paired_reference_vs_ga(
            reduced_service_selected_reps,
            reduced_service_results_df,
            "Reduced-Service",
            paired_metrics
        ),
        paired_reference_vs_ga(
            combined_selected_reps,
            combined_results_df,
            "Combined",
            paired_metrics
        )
    ],
    ignore_index=True
)

scenario_paired_ci.round(4)

```

Out[52]:

	Scenario	Period	Metric	N	Mean GA	Mean Reference	Mean Paired Diff (GA - Reference)	CI Lower	CI Upper	Statistically Supported Improvement
0	High-Demand	Morning	Mean Completed Wait	30	3.4767	3.6627	-0.1860	-0.2227	-0.1493	True
1	High-Demand	Morning	P95 Completed Wait	30	7.5200	9.0379	-1.5179	-1.6422	-1.3936	True
2	High-Demand	Morning	Peak Queue	30	21.8667	23.3667	-1.5000	-2.5084	-0.4916	True
3	High-Demand	Morning	Final Backlog	30	2.6667	2.8667	-0.2000	-0.3808	-0.0192	True
4	High-Demand	Evening	Mean Completed Wait	30	3.5002	3.6414	-0.1412	-0.1956	-0.0868	True
5	High-Demand	Evening	P95 Completed Wait	30	7.7810	7.8323	-0.0513	-0.3803	0.2778	False
6	High-Demand	Evening	Peak Queue	30	11.2000	13.9000	-2.7000	-3.3136	-2.0864	True
7	High-Demand	Evening	Final Backlog	30	1.5667	4.2667	-2.7000	-3.4282	-1.9718	True
8	Reduced-Service	Morning	Mean Completed Wait	30	3.8977	4.8501	-0.9524	-1.0344	-0.8704	True
9	Reduced-Service	Morning	P95 Completed Wait	30	9.0084	14.3864	-5.3779	-5.7168	-5.0391	True
10	Reduced-Service	Morning	Peak Queue	30	18.5667	24.5000	-5.9333	-7.3371	-4.5295	True
11	Reduced-Service	Morning	Final Backlog	30	2.1000	2.1667	-0.0667	-0.1614	0.0281	False

	Scenario	Period	Metric	N	Mean GA	Mean Reference	Mean Paired Diff (GA - Reference)	CI Lower	CI Upper	Statistically Supported Improvement
12	Reduced-Service	Evening	Mean Completed Wait	30	3.9677	4.3683	-0.4006	-0.5043	-0.2970	True
13	Reduced-Service	Evening	P95 Completed Wait	30	8.7662	11.9166	-3.1504	-3.6104	-2.6903	True
14	Reduced-Service	Evening	Peak Queue	30	8.8333	10.2667	-1.4333	-1.9853	-0.8814	True
15	Reduced-Service	Evening	Final Backlog	30	1.6000	3.2333	-1.6333	-2.0878	-1.1788	True
16	Combined	Morning	Mean Completed Wait	30	4.0105	4.8172	-0.8067	-0.8729	-0.7406	True
17	Combined	Morning	P95 Completed Wait	30	8.6150	14.2899	-5.6749	-5.9316	-5.4181	True
18	Combined	Morning	Peak Queue	30	24.3667	32.9000	-8.5333	-10.2536	-6.8131	True
19	Combined	Morning	Final Backlog	30	2.2667	2.8667	-0.6000	-0.8876	-0.3124	True
20	Combined	Evening	Mean Completed Wait	30	4.0025	4.3313	-0.3288	-0.4039	-0.2537	True
21	Combined	Evening	P95 Completed Wait	30	7.7989	11.7560	-3.9571	-4.3290	-3.5853	True
22	Combined	Evening	Peak Queue	30	12.3667	14.4000	-2.0333	-2.7019	-1.3648	True
23	Combined	Evening	Final Backlog	30	1.2333	4.2667	-3.0333	-3.8624	-2.2042	True

```
In [53]: # Summarize stop-level GA results across matched validation replications.
scenario_stop_level_summary = (
    selected_plan_stop_level
    .groupby(
```

```

        [
            "Scenario",
            "Period",
            "Stop Sequence"
        ]
    )
    [
        "Mean Wait",
        "P95 Wait",
        "Peak Queue",
        "Period-End Queue",
        "Final Backlog"
    ]
]
.mean()
.reset_index()
)

# Identify the highest mean peak-queue stop for each scenario and period.
highest_pressure_stops = (
    scenario_stop_level_summary
    .sort_values(
        [
            "Scenario",
            "Period",
            "Peak Queue"
        ],
        ascending=[
            True,
            True,
            False
        ]
    )
    .groupby(
        [
            "Scenario",
            "Period"
        ],
        as_index=False
    )
    .first()
)

```

```
)
highest_pressure_stops.round(3)
```

Out[53]:

	Scenario	Period	Stop Sequence	Mean Wait	P95 Wait	Peak Queue	Period-End Queue	Final Backlog
0	Combined	Evening	10	4.341	9.202	11.767	0.500	0.0
1	Combined	Morning	10	3.980	8.569	24.367	0.667	0.0
2	High-Demand	Evening	10	3.798	9.867	11.000	2.100	0.0
3	High-Demand	Morning	10	3.450	7.433	21.867	2.067	0.0
4	Reduced-Service	Evening	10	4.339	11.344	8.133	2.233	0.0
5	Reduced-Service	Morning	10	3.938	9.058	18.567	1.467	0.0

25. Export Genetic Algorithm Results

The output tables preserve the Baseline reproduction evidence, optimizer-run summaries, selected plans, full matched-replication results, safeguard checks, validation evidence, precision checks, paired confidence intervals, and stop-level summaries for High-Demand, Reduced-Service, and Combined.

```
In [54]: # Build reviewer-facing GA configuration evidence.
ga_configuration_df = pd.DataFrame([
    {
        "Scenario": name,
        "Morning Budget":
            ctx["service_budget"]["Morning"],
        "Evening Budget":
            ctx["service_budget"]["Evening"],
        "Minimum Hourly Trips":
            ctx["min_hourly_trips"],
        "Maximum Hourly Trips":
            ctx["max_hourly_trips"],
        "Morning Reference Plan":
            str(
```

```

        ctx[
            "reference_plan_lookup"
        ]["Morning"]
    ),
    "Evening Reference Plan":
        str(
            ctx[
                "reference_plan_lookup"
            ]["Evening"]
        ),
    "Search Replications":
        len(
            ctx[
                "search_replication_ids"
            ]
        ),
    "Validation Replications":
        len(
            ctx[
                "validation_replication_ids"
            ]
        ),
    "Optimizer Seeds":
        str(
            tuple(
                optimizer_seeds
            )
        ),
    "Unique Plans per Optimizer Run":
        evaluation_budget
}
for name, ctx in scenario_contexts.items()
])

# Define all GA output tables before writing them.
ga_exports = {
    "ga_core_baseline_runs.csv":
        ga_core_runs.assign(
            **{
                "Best Service Plan":
                    ga_core_runs[

```

```

        "Best Service Plan"
    ].astype(str)
    }
),
"ga_core_baseline_selected_plans.csv":
    ga_core_selected.assign(
        **{
            "Service Plan":
                ga_core_selected[
                    "Service Plan"
                ].astype(str)
        }
    ),
"ga_core_baseline_reproduction_check.csv":
    baseline_reproduction_check,
"ga_scenario_highdemand_runs.csv":
    high_demand_runs.assign(
        **{
            "Best Service Plan":
                high_demand_runs[
                    "Best Service Plan"
                ].astype(str)
        }
    ),
"ga_scenario_highdemand_selected_plans.csv":
    high_demand_selected.assign(
        **{
            "Service Plan":
                high_demand_selected[
                    "Service Plan"
                ].astype(str)
        }
    ),
"ga_scenario_reducedservice_runs.csv":
    reduced_service_runs.assign(
        **{
            "Best Service Plan":
                reduced_service_runs[
                    "Best Service Plan"
                ].astype(str)
        }
    )

```

```

    ),
    "ga_scenario_reducedservice_selected_plans.csv":
        reduced_service_selected.assign(
            **{
                "Service Plan":
                    reduced_service_selected[
                        "Service Plan"
                    ].astype(str)
            }
        ),
    "ga_scenario_combined_runs.csv":
        combined_runs.assign(
            **{
                "Best Service Plan":
                    combined_runs[
                        "Best Service Plan"
                    ].astype(str)
            }
        ),
    "ga_scenario_combined_selected_plans.csv":
        combined_selected.assign(
            **{
                "Service Plan":
                    combined_selected[
                        "Service Plan"
                    ].astype(str)
            }
        ),
    "ga_scenario_selected_plan_replications.csv":
        selected_plan_replications.assign(
            **{
                "Service Plan":
                    selected_plan_replications[
                        "Service Plan"
                    ].astype(str)
            }
        ),
    "ga_scenario_stop_level_results.csv":
        selected_plan_stop_level,
    "ga_scenario_stop_level_summary.csv":
        scenario_stop_level_summary,

```

```

    "ga_scenario_highest_pressure_stops.csv":
        highest_pressure_stops,
    "ga_scenario_validation_summary.csv":
        selected_validation_summary,
    "ga_scenario_precision_check.csv":
        selected_precision_check,
    "ga_scenario_paired_reference_vs_ga.csv":
        scenario_paired_ci,
    "ga_scenario_safeguards_check.csv":
        scenario_safeguards,
    "ga_scenario_configuration.csv":
        ga_configuration_df
}

for filename, dataframe in ga_exports.items():
    dataframe.to_csv(
        tables_dir / filename,
        index=False
    )

print(
    "GA table files exported:",
    len(ga_exports)
)

```

GA table files exported:
18

```

In [55]: # Verify that every expected GA output file exists and is non-empty.
ga_export_validation = pd.DataFrame([
    {
        "File":
            filename,
        "Exists":
            (
                tables_dir
                / filename
            ).exists(),
        "Rows":
            len(dataframe)
    }
]

```

```

        for filename, dataframe in ga_exports.items()
    ])

ga_export_validation[
    "Valid"
] = (
    ga_export_validation[
        "Exists"
    ]
    & (
        ga_export_validation[
            "Rows"
        ] > 0
    )
)

assert ga_export_validation[
    "Valid"
].all(), (
    "At least one expected GA output file "
    "is missing or empty."
)

print(
    "All expected GA outputs exported successfully:",
    ga_export_validation[
        "Valid"
    ].all()
)

ga_export_validation

```

All expected GA outputs exported successfully: True

Out[55]:

	File	Exists	Rows	Valid
0	ga_core_baseline_runs.csv	True	6	True
1	ga_core_baseline_selected_plans.csv	True	2	True
2	ga_core_baseline_reproduction_check.csv	True	2	True
3	ga_scenario_highdemand_runs.csv	True	6	True
4	ga_scenario_highdemand_selected_plans.csv	True	2	True
5	ga_scenario_reducedservice_runs.csv	True	6	True
6	ga_scenario_reducedservice_selected_plans.csv	True	2	True
7	ga_scenario_combined_runs.csv	True	6	True
8	ga_scenario_combined_selected_plans.csv	True	2	True
9	ga_scenario_selected_plan_replications.csv	True	180	True
10	ga_scenario_stop_level_results.csv	True	900	True
11	ga_scenario_stop_level_summary.csv	True	30	True
12	ga_scenario_highest_pressure_stops.csv	True	6	True
13	ga_scenario_validation_summary.csv	True	3	True
14	ga_scenario_precision_check.csv	True	12	True
15	ga_scenario_paired_reference_vs_ga.csv	True	24	True
16	ga_scenario_safeguards_check.csv	True	6	True
17	ga_scenario_configuration.csv	True	4	True

```
In [56]: # Save a compact manifest for reviewer navigation.
ga_output_manifest = pd.DataFrame([
    {
        "File":
            filename,
```

```

        "Directory":
            "output/tables",
        "Purpose":
            (
                "Baseline reproduction evidence"
                if filename.startswith(
                    "ga_core_"
                )
                else "GA scenario result or validation evidence"
            )
    }
    for filename in ga_exports
])

ga_output_manifest.to_csv(
    tables_dir
    / "ga_output_manifest.csv",
    index=False
)

print(
    "GA output manifest saved:",
    tables_dir
    / "ga_output_manifest.csv"
)

```

GA output manifest saved: C:\Users\User\Documents\GitHub\m66-passenger-queue-simulation\MODESIM_Project\output\tables\ga_output_manifest.csv

Pre-Final GA Comparison Figures

The following figures summarize the final matched reference-versus-GA results for the three stress scenarios. They are generated from `ga_scenario_paired_reference_vs_ga.csv`, which contains the 30-replication paired comparisons used for the statistical evaluation.

Each figure presents the non-optimized reference mean beside the selected GA-plan mean. Morning and Evening are shown separately for High-Demand, Reduced-Service, and Combined conditions.

In [57]: *# Generate reviewer-facing reference-versus-GA figures from the final
paired comparison table. This cell does not rerun or alter the GA search.*

```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

working_dir = Path.cwd()
project_candidates = [
    working_dir,
    working_dir.parent,
    working_dir / "MODESIM_Project",
]

figure_project_dir = next(
    (
        path
        for path in project_candidates
        if (path / "code").exists() and (path / "output").exists()
    ),
    None,
)

if figure_project_dir is None:
    raise FileNotFoundError(
        "Could not locate MODESIM_Project for GA figure generation."
    )

figure_tables_dir = figure_project_dir / "output" / "tables"
figure_output_dir = figure_project_dir / "output" / "figures"
figure_output_dir.mkdir(parents=True, exist_ok=True)

paired_results_file = (
    figure_tables_dir / "ga_scenario_paired_reference_vs_ga.csv"
)

if not paired_results_file.exists():
    raise FileNotFoundError(
        "Missing ga_scenario_paired_reference_vs_ga.csv. "
```

```

        "Run the GA result-export section first."
    )

paired_results = pd.read_csv(paired_results_file)

required_columns = {
    "Scenario",
    "Period",
    "Metric",
    "Mean GA",
    "Mean Reference",
}

missing_columns = required_columns.difference(paired_results.columns)
if missing_columns:
    raise ValueError(
        "Paired GA table is missing required columns: "
        + ", ".join(sorted(missing_columns))
    )

scenario_order = ["High-Demand", "Reduced-Service", "Combined"]
period_order = ["Morning", "Evening"]

figure_specs = [
    (
        "Mean Completed Wait",
        "Mean Completed Wait (minutes)",
        "ga_reference_vs_optimized_mean_wait.png",
    ),
    (
        "P95 Completed Wait",
        "P95 Completed Wait (minutes)",
        "ga_reference_vs_optimized_p95_wait.png",
    ),
    (
        "Peak Queue",
        "Peak Queue (passengers)",
        "ga_reference_vs_optimized_peak_queue.png",
    ),
    (
        "Final Backlog",

```

```

        "Final Backlog (passengers)",
        "ga_reference_vs_optimized_final_backlog.png",
    ),
]

generated_ga_figures = []

order_lookup = {
    (scenario, period): index
    for index, (scenario, period) in enumerate(
        (scenario, period)
        for scenario in scenario_order
        for period in period_order
    )
}

for metric, y_label, filename in figure_specs:
    metric_rows = paired_results[
        paired_results["Metric"].eq(metric)
    ].copy()

    metric_rows = metric_rows[
        metric_rows["Scenario"].isin(scenario_order)
        & metric_rows["Period"].isin(period_order)
    ].copy()

    if len(metric_rows) != 6:
        raise ValueError(
            f"Expected 6 stress-scenario rows for {metric}, "
            f"found {len(metric_rows)}."
        )

    metric_rows["Plot Order"] = metric_rows.apply(
        lambda row: order_lookup[(row["Scenario"], row["Period"])],
        axis=1,
    )
    metric_rows = metric_rows.sort_values("Plot Order").reset_index(drop=True)

    labels = [
        f"{scenario}\n{period}"
        for scenario, period in zip(

```

```

        metric_rows["Scenario"],
        metric_rows["Period"],
    )
]

x_positions = list(range(len(metric_rows)))
bar_width = 0.38

fig, ax = plt.subplots(figsize=(11, 6))

reference_positions = [
    position - bar_width / 2 for position in x_positions
]
ga_positions = [
    position + bar_width / 2 for position in x_positions
]

ax.bar(
    reference_positions,
    metric_rows["Mean Reference"],
    width=bar_width,
    label="Reference",
)
ax.bar(
    ga_positions,
    metric_rows["Mean GA"],
    width=bar_width,
    label="GA Selected Plan",
)

ax.set_title(f"Reference vs GA Selected Plan: {metric}")
ax.set_ylabel(y_label)
ax.set_xticks(x_positions)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis="y", alpha=0.25)

fig.tight_layout()

output_path = figure_output_dir / filename
fig.savefig(output_path, dpi=200, bbox_inches="tight")

```

```

plt.close(fig)

generated_ga_figures.append(output_path)

manifest_path = figure_tables_dir / "ga_output_manifest.csv"

if not manifest_path.exists():
    raise FileNotFoundError(
        "Missing ga_output_manifest.csv. Run the GA export section first."
    )

ga_manifest = pd.read_csv(manifest_path)

expected_manifest_columns = {"File", "Directory", "Purpose"}
if not expected_manifest_columns.issubset(ga_manifest.columns):
    raise ValueError(
        "ga_output_manifest.csv has an unexpected column structure."
    )

figure_manifest_rows = pd.DataFrame(
    [
        {
            "File": path.name,
            "Directory": "output/figures",
            "Purpose": "Reviewer-facing reference-versus-GA comparison figure",
        }
        for path in generated_ga_figures
    ]
)

ga_manifest = pd.concat(
    [ga_manifest, figure_manifest_rows],
    ignore_index=True,
).drop_duplicates(
    subset=["File", "Directory"],
    keep="last",
)

ga_manifest.to_csv(manifest_path, index=False)

missing_figures = [

```

```

    str(path)
    for path in generated_ga_figures
    if not path.exists() or path.stat().st_size == 0
]

if missing_figures:
    raise RuntimeError(
        "One or more GA comparison figures were not created:\n"
        + "\n".join(missing_figures)
    )

print("GA comparison figures created successfully:")
for path in generated_ga_figures:
    print(f"- {path.name}")

print("GA output manifest updated with reviewer-facing figures: True")

```

GA comparison figures created successfully:

- ga_reference_vs_optimized_mean_wait.png
- ga_reference_vs_optimized_p95_wait.png
- ga_reference_vs_optimized_peak_queue.png
- ga_reference_vs_optimized_final_backlog.png

GA output manifest updated with reviewer-facing figures: True